Final Year Project Report

BS (COPMPUTER SCIENCE/ SOFTWARE ENGINEERING)

TALENTSAVER

SUBMITTED BY:

Shoaib Ahmed	021-22-62139
Abdul Basit Memon	021-20-53424
Muhammad Muzammil Naveed	021-20-53065
Muhammad Imtiaz Ali	021-20-54007

<Supervisor>

SIR MAAZ AHMED

<Coordinator>

DR. RIZWAN MUNIR

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Student Name 1

SHOAIB AHMED

_____

Student Name 2

ABDUL BASIT MEMON

_____

Student Name 3

MUHAMMAD MUZAMMIL NAVEED

_____

Student Name 4

MUHAMMAD IMTIAZ ALI

_____

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS/SE) "Project title" was developed by **SHOAIB AHMED (021-22-62139)**, **ABDUL BASIT MEMON (021-20-53424), MUHAMMAD MUZAMMIL NAVEED (021-20-53065), MUHAMMAD IMTIAZ ALI (021-20-54007)** under the supervision of "**SIR MAAZ AHMED**" and co supervisor "**DR RIZWAN MUNIR**" and that in (their/his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences/software Engineering.

Supervisor

Examiner

Head of Department

(Department of Computer Science/Software Engineering/AI)

EXECUTIVE SUMMARY

TalentSaver

TalentSaver is an advanced machine learning-based solution designed to address the challenge of employee replacement due to attrition. In the fast-paced work environment, replacing employees who leave is critical for maintaining organizational stability and productivity. TalentSaver helps organizations identify employees who are likely to leave and provides suitable replacements, ensuring a seamless transition and minimizing disruptions.

The system integrates machine learning models that predict employee attrition and then suggest the best possible replacement from within the organization. Key features include the ability to input employee data, process it efficiently, and deliver real-time predictions of both attrition and potential replacements. The system is equipped with mobile responsiveness, cross-browser compatibility, and user authentication to ensure security.

TalentSaver's data preprocessing capabilities ensure the accuracy and quality of input data, making predictions more reliable. By automating the replacement process and offering data-driven insights, organizations can reduce the cost of recruitment, maintain workforce continuity, and improve employee satisfaction.

The machine learning models used by TalentSaver are continuously refined, learning from new data to provide the most accurate predictions. Its user-friendly interface and real-time capabilities make TalentSaver a comprehensive tool for managing employee attrition and ensuring effective workforce planning.

ACKNOWLEDGEMENT

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task. We are greatly indebted to our project supervisor "**SIR MAAZ AHMED**" and our Co-Supervisor "**DR RIZWAN MUNIR**". Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work. And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

ABBREVIATIONS

HR	HUMAN RESOURCE
MLA	MACHINE LEARNING ALGORITHMS
JS	JOB SATISFACTION
RF	RANDOM FOREST
WF	WORK FORCE
SVM	SUPPORT VECTOR MACHINE
KPI	KEY PERFOMANCE INDICATOR

LIST OF TABLE

Table of Contents

DECLARATION ii
CERTIFICATE OF APPROVAL iii
EXECUTIVE SUMMARY iv
ACKNOWLEDGEMENT v
ABBREVIATIONS 12
Chapter # 01 Project Introduction 2
1.1 Project Background 2
1.2 Objective, Benefits and Deliverables 3
1.2.1 Develop a Web Application Framework: 3
1.2.2 Implement Machine Learning Integration: 3
1.2.3 Design an Intuitive User Interface (UI): 3
1.2.4 Incorporate User Authentication and Authorization: 3
1.2.5 Enable Data Input and Preprocessing: 3
1.2.6 Provide Prediction Results in Real-Time: 3
1.2.7 Conduct Cross-Browser Compatibility Testing: 4
1.2.8 Optimize for Mobile Responsiveness: 4
1.2.9 Implement Error Handling and Logging: 4
1.2.10 Deploy the Web Application: 4
1.2.11 Monitor Web Application Performance: 4
1.3 Key Benefits 5
1.3.1 In-Depth Data Analysis: 5
1.3.2 Predictive Modeling Capability: 5
1.3.3 Cost-Efficiency: 5
1.3.4 Enhanced Employee Engagement: 5
1.3.5 Cultivating a Positive Organizational Culture: 5
1.3.6 Strategic Human Resource Planning: 5
1.4 Beneficiaries 6

1.4.1 Human Resources Professionals: 6
1.4.2 Organizational Leadership: 6
1.4.3 Current Employees: 6
1.4.4 Recruitment Teams: 6
1.4.5 Shareholders/Investors: 6
1.4.6 Academic Community: 6
1.5 Deliverables 7
1.5.1 Project Proposal: 7
1.5.2 Data Collection and Preprocessing: 7
1.5.3 Feature Engineering: 7
1.5.4 Model Development: 7
1.5.5 Model Evaluation: 7
1.5.6 Deployment: 7
1.5.7 Documentation: 7
1.5.8 Presentation and Visualization: 7
1.5.9 Training and Support: 7
1.5.10 Feedback Mechanism: 8
1.5.11 Final Report: 8
1.6 Implementation Plan 9
1.6.1 Development / Research Methodology 9
1.7 Expected Outcomes 11
1.8 Project Schedule / Milestone Chart 12
Chapter # 02 Related System Analysis/Literature Review 14
2.1 Akkio 14
2.1.1 INTRODUCTION: 14
2.1.2 WORKING PRINCIPLE: 14
2.1.3 CONCLUSION: 15
2.1.4 SCREENSHOT: 16
2.2 Synvert ClearPeaks 17
2.2.1 INTRODUCTION: 17
2.2.2 WORKING PRINCIPLE: 17
2.2.3 CONCLUSION: 17
2.2.4 SCREENSHOT: 18

2.3 Hindawi 19
2.3.1 INTRODUCTION: 19
2.3.2 WORKING PRINCIPLE: 19
2.3.3 CONCLUSION: 20
2.3.4 SCREENSHOT: 21
2.4 MDPI 22
2.4.1 INTRODUCTION: 22
2.4.2 WORKING PRINCIPLE: 22
2.4.3 CONCLUSION: 23
2.4.4 SCREENSHOT: 24
2.5 Global Scientific JOURNALS 25
2.5.1 INTRODUCTION: 25
2.5.2 WORKING PRINCIPLE: 25
2.5.3 CONCLUSION: 25
2.5.4 SCREENSHOT: 26
2.6 MAJOR DIFFERENCES BETWEEN OUR PROJECT AND OTHERS PROJECTS: 27
Chapter # 03 Research Approach 29
3.1 Data Set 29
3.1.2 Sample Dataset Structure 29
3.2 Dataset Sources 31
3.3 Algorithms Parameter Settings 32
3.3.1 Common Algorithms and Their Parameters 32
3.4 Milestone Chart 35
Chapter # 04 Requirement Analysis 36
4.1 Functional Requirements: 36
4.2 Nonfunctional Requirements: 37
4.3 USE CASE DIAGRAM 38
4.3.1 USE CASE DESCRIPTION 39
4.4 UML CLASS DIAGRAM 40
4.5 HIGHLY LEVEL ARCHITECTURE 41
Chapter # 05 Design and Architecture 42
5.1 Architectural design 42
5.1.1 Class Diagram 42

5.1.2 Class Diagram 43
5.1.3 State Transition Diagram 44
5.1.4 State Transition Diagram 45
5.1.5 Data Flow Diagram (DFD) 46
5.1.6 Data Flow Diagram (DFD): 47
5.1.7 Unique Framework for Employee Attrition Prediction 48
5.1.8 Framework Components: 48
5.1.9 Flowchart of Enhanced Attrition Prediction Framework (EAPF): 52
5.2 Data Design 53
5.3 Human interface design (product based or hardware system) 56
5.4 How the User Will Use the System to Complete All Expected Features 58
5.5 Feedback Information Displayed for the User 59
5.6 USER INTERFACE DESIGN – TEMPLATE 60
5.6.1 Usability Heuristics for User Interface Design 65
Project Title: TalentSaver 65
5.6.2 Clarity, Familiarity, User Control, and Flexibility 65
5.7 Hierarchy and Negative Space 66
5.8 Accessibility 67
5.9 Summary 67
Chapter # 06 Implementation 68
6.1 Algorithms 68
6.2 Explanation of Main Features: 70
Chapter # 07 Testing and Evaluation 71
7.1 Testing Phase 71
Test Case Id: 1.1 71
Test Case Id: 1.2 71
Test Case Id: 1.3 72
Test Case Id: 1.4 72
Test Case Id: 1.5 73
Test Case Id: 1.6 73
Test Case Id: 1.7 74
Test Case Id: 1.8 74
Test Case Id: 1.9 75

Test Case Id: 1.10 75
Test Case Id: 1.11 76
Test Case Id: 1.12 76
Test Case Id: 1.13 77
Test Case Id: 1.14 77
Test Case Id: 1.15 78
Chapter # 08 Conclusion and Future Work 79
8.1 Conclusion 79
8.2 System Limitations and Challenges: 80
8.3 Future Work 81
8.5 Conclusion: 83
Chapter # 9 Complete Code Of Final Year Project 84
9.1 Backend 84
9.1.1 Main source code 84
9.1.2 App.py 93
9.1.3 Requirementst.txt 100
9.2 Frontend 101
9.2.1 Index 101
9.2.2 Login 104
9.2.3 Signup 107
9.2.4 Users 110
9.2.5 About 112
9.2.6 Add_Employee 117
9.2.7 Navbar 119
9.2.8 Results 121
References 126

List of Figures

Figures **Page #**

Figure: 1.0---------- 12
Figure: 1.1---------- 12
Figure: 2.1.0, 2.1.1---------- 16
Figure: 2.1.2, 2.1.3---------- 16
Figure: 2.1.4, 2.1.5---------- 16
Figure: 2.2.0, 2.2.1---------- 18
Figure: 2.2.2, 2.2.3---------- 18
Figure: 2.3.0, 2.3.1---------- 21
Figure: 2.3.2, 2.3.3---------- 21
Figure: 2.3.4, 2.3.5---------- 21
Figure: 2.4.1 ---------- 24
Figure: 2.4.2, 2.4.3---------- 24
Figure: 2.4.4, 2.4.5---------- 24
Figure: 2.5.0, 2.5.1---------- 27
Figure: 2.5.2, 2.5.3---------- 27
Figure: 4.1---------- 40
Figure: 4.2---------- 42
Figure: 4.3---------- 43
Figure: 5.1---------- 45
Figure: 5.2---------- 47
Figure: 5.3---------- 49
Figure: 5.4---------- 54
Figure: 5.4.1---------- 62
Figure: 5.4.2---------- 63
Figure: 5.4.3---------- 64
Figure: 5.4.4 ---------- 65
Figure: 5.4.5 ---------- 66

Chapter # 01 Project Introduction

1.1 Project Background

Our project as TalentSaver stands at the forefront of innovation in employee attrition management, leveraging cutting-edge machine learning models to predict and prevent talent loss. Distinguishing itself from conventional projects, TalentSaver boasts a user-friendly interface, real-time prediction capabilities, and a focus on mobile responsiveness. Unlike typical attrition systems, our project offers a comprehensive analysis of employee engagement metrics, delving into the root causes of dissatisfaction and providing actionable insights. The integration of personalized user feedback and continuous improvement mechanisms sets TalentSaver apart, ensuring adaptability to evolving organizational needs. By addressing challenges such as talent shortage, recruitment costs, and productivity disruptions, TalentSaver emerges as a holistic solution, promising to reshape the landscape of talent management. With a commitment to excellence and a vision for a proactive approach to employee retention, TalentSaver heralds a new era in talent management practices.

1.2 Objective, Benefits and Deliverables

1.2.1 Develop a Web Application Framework:

Objective: To develop a user-friendly web application framework for hosting the attrition prediction system.

Expected Result: A functional web interface accessible to users, providing a platform for interacting with the machine learning models.

1.2.2 Implement Machine Learning Integration:

Objective: To integrate machine learning models into the web application for real-time predictions.

Expected Result: Seamless integration allowing users to input relevant data and receive attrition predictions through the web interface.

1.2.3 Design an Intuitive User Interface (UI):

Objective: To design an intuitive and visually appealing UI for the web application.

Expected Result: A user interface that is easy to navigate, with clear input forms and visualizations of attrition predictions.

1.2.4 Incorporate User Authentication and Authorization:

Objective: To implement user authentication and authorization features for secure access to the web application.

Expected Result: A system that ensures only authorized personnel can access and utilize the attrition prediction functionalities.

1.2.5 Enable Data Input and Preprocessing:

Objective: To develop data input functionalities allowing users to input employee data for prediction.

Expected Result: An interface enabling users to enter employee information, with built-in preprocessing to handle data quality and consistency.

1.2.6 Provide Prediction Results in Real-Time:

Objective: To implement real-time prediction capabilities, delivering instant results to users.

Expected Result: Users receive attrition predictions promptly upon submitting relevant employee data through the web interface.

1.2.7 Conduct Cross-Browser Compatibility Testing:

Objective: To ensure the web application functions seamlessly across various web browsers.

Expected Result: Verification that the web application performs consistently and accurately on popular browsers (e.g., Chrome, Firefox and Safari).

1.2.8 Optimize for Mobile Responsiveness:

Objective: To optimize the web application for mobile devices, ensuring a responsive design.

Expected Result: A user-friendly experience on mobile devices, with the web application adapting to different screen sizes.

1.2.9 Implement Error Handling and Logging:

Objective: To implement robust error handling mechanisms and logging features.

Expected Result: The system effectively logs errors and provides informative error messages to users when issues arise.

1.2.10 Deploy the Web Application:

Objective: To deploy the attrition prediction web application to a production environment.

Expected Result: The web application is live and accessible to users, hosted on a reliable server.

1.2.11 Monitor Web Application Performance:

Objective: To establish a monitoring system for tracking web application performance.

Expected Result: Continuous monitoring ensuring the web application remains responsive and performs optimally under various usage scenarios.

1.3 Key Benefits

1.3.1 In-Depth Data Analysis:

This project offers a thorough examination of employee attrition data, providing valuable insights into the underlying causes of turnover.

1.3.2 Predictive Modeling Capability:

Through advanced analytics, the project has the potential to develop models that predict attrition, allowing organizations to take proactive measures to retain key personnel.

1.3.3 Cost-Efficiency:

By identifying and addressing the root causes of attrition, the project aids in reducing recruitment and training costs associated with employee turnover.

1.3.4 Enhanced Employee Engagement:

The findings of the project can be instrumental in crafting strategies that improve employee engagement, job satisfaction, and overall well-being in the workplace.

1.3.5 Cultivating a Positive Organizational Culture:

Addressing attrition-related issues contributes to fostering a healthier and more positive organizational culture.

1.3.6 Strategic Human Resource Planning:

The project equips HR departments with insights to develop strategic plans for talent management, succession planning, and maintaining a stable workforce.

1.4 Beneficiaries

1.4.1 Human Resources Professionals:

HR practitioners can leverage the insights gained from the project to implement targeted strategies for employee retention, talent development, and overall workforce management.

1.4.2 Organizational Leadership:

Executives and leaders can make informed decisions based on the project's findings, formulating policies and initiatives that nurture a stable and satisfied workforce.

1.4.3 Current Employees:

The project indirectly benefits existing employees by contributing to a more supportive work environment, potentially reducing turnover and fostering a positive workplace culture.

1.4.4 Recruitment Teams:

Teams responsible for hiring can gain valuable insights to refine their recruitment processes, attracting candidates better aligned with the organization.

1.4.5 Shareholders/Investors:

A stable workforce positively impacts the financial stability of the organization, potentially increasing confidence and satisfaction among shareholders and investors.

1.4.6 Academic Community:

If the project is part of academic research, its findings contribute to the academic understanding of HR management and organizational behavior.

1.5 Deliverables

1.5.1 Project Proposal:

Outline the objectives, methodology, timeline, and expected outcomes of the project.

1.5.2 Data Collection and Preprocessing:

a. Gather relevant employee data from HR systems, surveys, and other sources.
b. Clean, preprocess, and format the data to ensure quality and consistency.

1.5.3 Feature Engineering:

a. Identify and select features that are predictive of employee attrition.

1.5.4 Model Development:

a. Choose appropriate machine learning algorithms (e.g., logistic regression, random forest, neural networks).
b. Train and optimize the model using historical data.

1.5.5 Model Evaluation:

a. Assess the performance of the trained model using evaluation metrics such as accuracy, precision, recall, and F1 score.
b. Validate the model using techniques like cross-validation or holdout validation.

1.5.6 Deployment:

a. Integrate the trained model into HR systems or develop a user-friendly interface for HR professionals to input data and receive predictions.
b. Ensure scalability, reliability, and security of the deployed solution.

1.5.7 Documentation:

a. Prepare comprehensive documentation covering data sources, preprocessing steps, feature selection rationale, model architecture, hyper parameters, and evaluation results.
b. Include user guides and instructions for deploying and using the model.

1.5.8 Presentation and Visualization:

a. Create visualizations to illustrate key findings, such as feature importance, model performance metrics, and prediction results.
b. Prepare a presentation summarizing the project objectives, methodology, findings, and recommendations.

1.5.9 Training and Support:

a. Provide training sessions for HR professionals on using the model effectively.

b. Offer ongoing support and troubleshooting assistance post-deployment.

1.5.10 Feedback Mechanism:

a. Establish a mechanism for collecting feedback from users to iteratively improve the model.

1.5.11 Final Report:

a. Compile all project deliverables, including the proposal, documentation, presentation slides, and any additional materials, into a final report for stakeholders.

1.6 Implementation Plan

1.6.1 Development / Research Methodology

The development or research methodology for an employee attrition prediction project involves several key steps. Here's a structured approach:

i. **Problem Definition:**
   Clearly define the problem statement, objectives, and scope of the project. Identify why predicting employee attrition is essential for the organization.

ii. **Data Collection:**
   Gather relevant data from HR systems, employee databases, surveys, performance evaluations, and other sources. Ensure data privacy and compliance with regulations like GDPR.

iii. **Data Preprocessing:**
   Clean the data to handle missing values, outliers, and inconsistencies. Normalize or standardize numerical features, encode categorical variables, and perform any necessary transformations.

iv. **Exploratory Data Analysis (EDA):**
   Explore the dataset to understand its characteristics, distributions, correlations, and potential patterns related to attrition. Visualize data using plots, histograms, and heat maps.

v. **Feature Selection and Engineering:**
   Identify features that are predictive of employee attrition through statistical analysis, domain knowledge, and correlation studies. Create new features or transformations to enhance predictive power.

vi. **Model Selection:**
   Choose appropriate machine learning algorithms based on the nature of the problem (e.g., classification for binary attrition prediction). Consider algorithms like logistic regression, decision trees, random forests, support vector machines, or neural networks.

vii. **Model Training:**

Split the dataset into training and validation sets. Train the selected models using the training data while tuning hyper parameters to optimize performance. Utilize techniques like cross-validation for robustness.

viii. **Model Evaluation:**

Evaluate model performance using metrics such as accuracy, precision, recall, F1 score, and area under the ROC curve. Compare multiple models to select the best-performing one.

ix. **Deployment:**

Integrate the trained model into HR systems or develop a user-friendly interface for HR professionals to input data and receive predictions. Ensure scalability, security, and reliability of the deployed solution.

x. **Monitoring and Maintenance:**

Establish a system for monitoring model performance in production, detecting drift, and retraining the model periodically with new data. Implement feedback mechanisms for continuous improvement.

xi. **Documentation and Reporting:**

Document the entire process, including data sources, preprocessing steps, feature selection rationale, model architecture, hyper parameters, evaluation results, and deployment instructions. Prepare a comprehensive report for stakeholders.

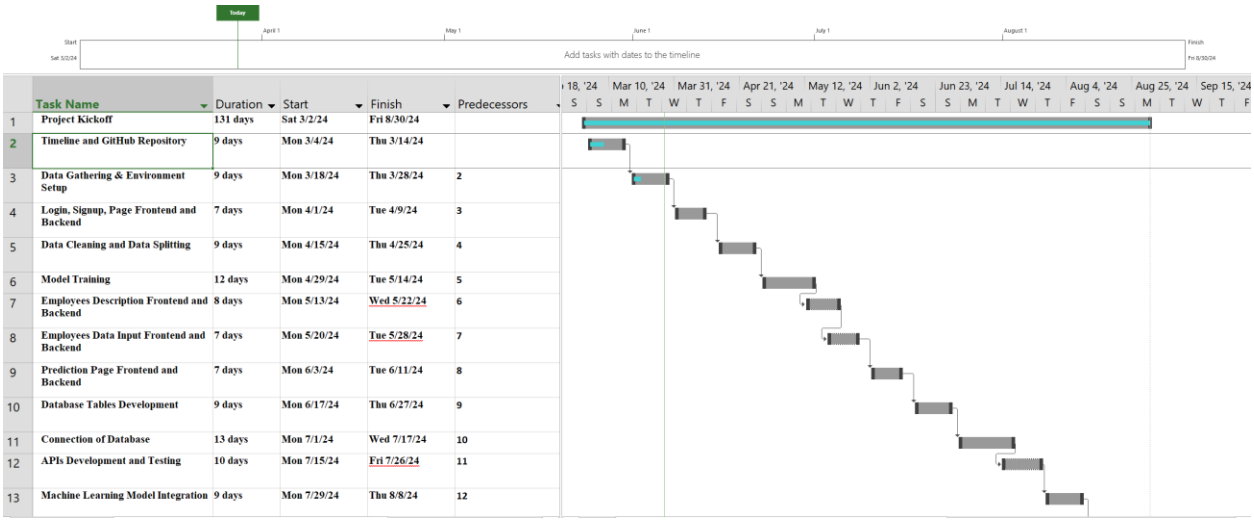
xii. **Ethical Considerations:**

Address ethical concerns related to employee privacy, fairness, bias, and transparency throughout the project lifecycle. Ensure compliance with legal and regulatory requirements.

1.7 Expected Outcomes

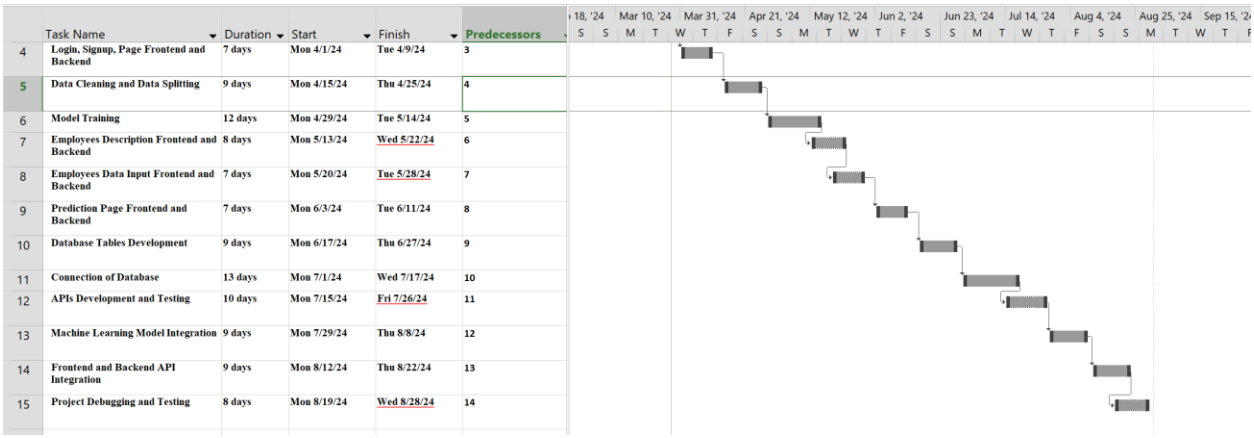
The envisioned outcomes include the development of highly accurate machine learning models adept at reliably predicting employee attrition. Furthermore, the project aims to establish a user-friendly web interface, seamlessly integrating machine learning algorithms for real-time predictions. The focus is on designing an intuitive user interface simplifying data input and presenting clear prediction results. Users can anticipate instantaneous delivery of attrition predictions through the web interface. The application will perform consistently across various web browsers. Ensuring a responsive experience. Incorporating effective error handling, logging mechanisms, and informative user messages will enhance user experience and system reliability.

1.8 Project Schedule / Milestone Chart



	Task Name	Duration	Start	Finish	Predecessors
1	Project Kickoff	131 days	Sat 3/2/24	Fri 8/30/24	
2	Timeline and GitHub Repository	9 days	Mon 3/4/24	Thu 3/14/24	
3	Data Gathering & Environment Setup	9 days	Mon 3/18/24	Thu 3/28/24	2
4	Login, Signup, Page Frontend and Backend	7 days	Mon 4/1/24	Tue 4/9/24	3
5	Data Cleaning and Data Splitting	9 days	Mon 4/15/24	Thu 4/25/24	4
6	Model Training	12 days	Mon 4/29/24	Tue 5/14/24	5
7	Employees Description Frontend and Backend	8 days	Mon 5/13/24	Wed 5/22/24	6
8	Employees Data Input Frontend and Backend	7 days	Mon 5/20/24	Tue 5/28/24	7
9	Prediction Page Frontend and Backend	7 days	Mon 6/3/24	Tue 6/11/24	8
10	Database Tables Development	9 days	Mon 6/17/24	Thu 6/27/24	9
11	Connection of Database	13 days	Mon 7/1/24	Wed 7/17/24	10
12	APIs Development and Testing	10 days	Mon 7/15/24	Fri 7/26/24	11
13	Machine Learning Model Integration	9 days	Mon 7/29/24	Thu 8/8/24	12

Figure: 1.0



	Task Name	Duration	Start	Finish	Predecessors
4	Login, Signup, Page Frontend and Backend	7 days	Mon 4/1/24	Tue 4/9/24	3
5	Data Cleaning and Data Splitting	9 days	Mon 4/15/24	Thu 4/25/24	4
6	Model Training	12 days	Mon 4/29/24	Tue 5/14/24	5
7	Employees Description Frontend and Backend	8 days	Mon 5/13/24	Wed 5/22/24	6
8	Employees Data Input Frontend and Backend	7 days	Mon 5/20/24	Tue 5/28/24	7
9	Prediction Page Frontend and Backend	7 days	Mon 6/3/24	Tue 6/11/24	8
10	Database Tables Development	9 days	Mon 6/17/24	Thu 6/27/24	9
11	Connection of Database	13 days	Mon 7/1/24	Wed 7/17/24	10
12	APIs Development and Testing	10 days	Mon 7/15/24	Fri 7/26/24	11
13	Machine Learning Model Integration	9 days	Mon 7/29/24	Thu 8/8/24	12
14	Frontend and Backend API Integration	9 days	Mon 8/12/24	Thu 8/22/24	13
15	Project Debugging and Testing	8 days	Mon 8/19/24	Wed 8/28/24	14

Figure: 1.1

S. No	Elapsed time since start of the project	Milestone	Deliverable
1.	Week 01	Timeline and GitHub Repository	Complete…
2.	Week 02	Data Gathering & Environment Setup	Complete…
3.	Week 03	Login, Signup, Page Frontend and Backend	Complete…
4.	Week 04	Data Cleaning and Data Splitting	Complete…
5.	Week 05	Model Training	Complete…
6.	Week 06	Employees Description Frontend and Backend	Working…
7.	Week 07	Employees Data Input Frontend and Backend	Working…
8.	Week 08	Prediction Page Frontend and Backend	Pending…
9.	Week 09	Database Tables Development	Pending…
10.	Week 10	Connection of Database	Pending…
11	Week 11	APIs Development and Testing	Pending…
12	Week 12	Machine Learning Model Integration	Pending…
13	Week 13	Frontend and Backend API Integration	Pending…
14	Week 14	Project Debugging and Testing	Pending…

Final deliverable of the project (please tick of the following)		
☐ Hardware system	● Software system	☐ HW/SW integrated system
☐ Software simulation results	☐ Comparative study	● Web Application
☐ Simulator design	☐ Mobile Application	☐ Other (please specify)

Chapter # 02 Related System Analysis/Literature Review

2.1 Akkio

2.1.1 INTRODUCTION:

The Akkio API is designed to be a 1:1 parity of our web interface. Anything that can be achieved visually can also be called in the API. This section provides an introduction on how to set up and test the setup of the API as well as a list of the commands available for use with example syntax. For beginners, we recommend running the quick start to ensure that your setup is correct, as it is a solution on rails that will ensure you are starting from a good state. If you have any issues with commands that may need to be added to the API or the documentation, we are happy to work with you to either point you to the right place or implement any required updates. Contact our support through our in-app chat or at support@akkio.com.

2.1.2 WORKING PRINCIPLE:

Churn is a pervasive problem for companies in all industries. It is costly and can be difficult to predict and prevent. For these reasons, it has been called the "silent killer" of SaaS businesses. In fact, a 1% increase in churn can result in a 27% decrease in profitability, according to Bain & Company. There are many reasons why customers may choose to leave a company, including poor customer service, high prices, or a lack of features. Fortunately, there are some steps that companies can take to reduce their churn rate.

In this section, we will demonstrate how Akkio can be used to set up an end to end machine learning workflow for churn prediction.

i. Data collection
ii. Creating and training your model
iii. Evaluate model performance
iv. Deployment

2.1.3 CONCLUSION:

As AI becomes more commonplace in business, organizations are looking for ways to get on board without hiring data scientists and programmers. Akkio offers a no-code AI platform that makes it easy for anyone to build and deploy intelligent applications. With Akkio, you don't need to be a data scientist or programmer to use AI.Akkio's simple drag-and-drop interface makes it easy to build custom models without writing a single line of code. You can easily incorporate pre-built algorithms or build your own models using our library of data science functions. Akkio takes care of all the heavy lifting, so you can focus on getting the most value from your data. Akkio is also fully integrated with popular business tools like Salesforce and Snowflake. This means you can easily bring AI into your existing workflows and get started quickly.

2.1.4 SCREENSHOT:

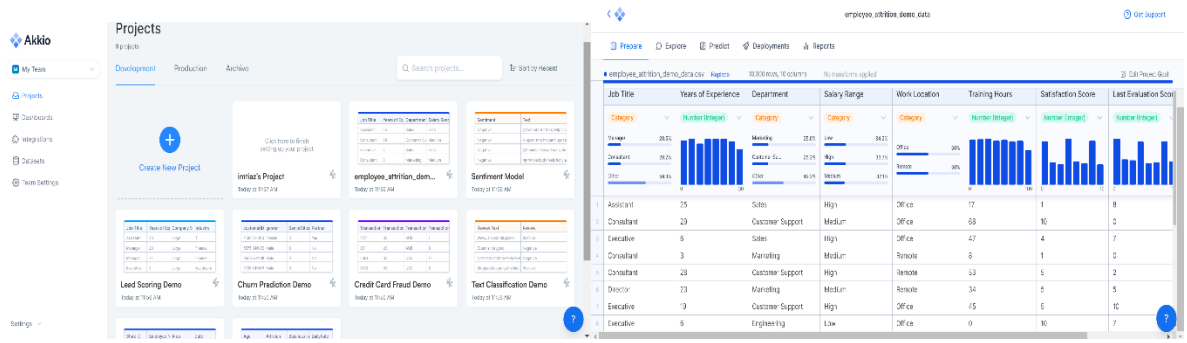


Figure 2.1.0, Figure 2.1.1

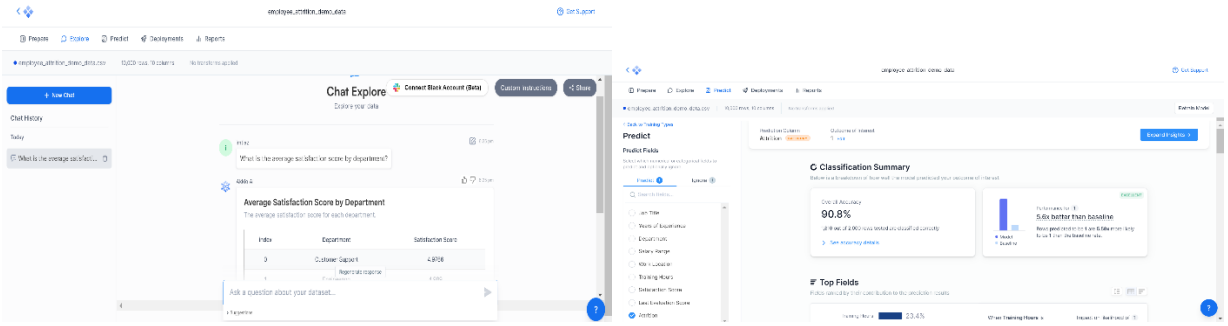


Figure 2.1.2, Figure 2.1.3

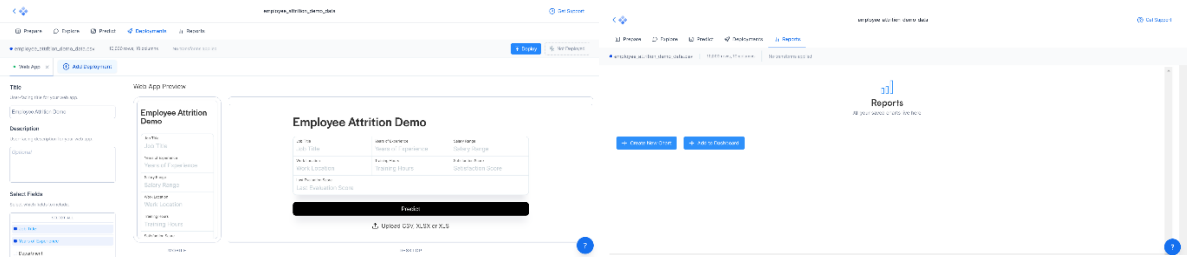


Figure 2.1.4, Figure 2.1.5

2.2 Synvert ClearPeaks

Predicting employee attrition with Machine learning using KNIME

2.2.1 INTRODUCTION:

We are already seeing how Advanced Analytics (AA) is helping to increase profits in many companies. However, some businesses are late in the adoption of AA due to a lack of information about its power and all the ways it can be used, while others are trying to adopt AA but are failing for various reasons. In most cases, the non-adoption or failed adoption is not due to a technology gap, since almost any organization can now access AA technology, be it through open-source languages such as R or Python, or through tools that ease Artificial Intelligence (AI) operations such as KNIME, Alteryx, Dataiku, Rapid miner, Oracle Data Miner, H2O, Azure Machine Learning Studio etc. ClearPeaks is already helping many businesses to adopt AA, and in this blog article we will review, as an illustrative example, an AA use case involving Machine Learning (ML) techniques. The success of the digital transformation and AA adoption in any business depends on the participation of most of its departments.

2.2.2 WORKING PRINCIPLE:

Today we will show you how Advanced Analytics with Machine Learning can help HR departments to retain talent. Employee attrition refers to the percentage of workers who leave an organization and are replaced by new employees; a high rate of attrition in an organization leads to increased recruitment, hiring and training costs.

2.2.3 CONCLUSION:

In this blog article we have detailed the various steps when implementing an advanced analytics use case in HR, employee attrition. We used the open-source tool KNIME to prepare the data, train different models, compare them and choose the best. With the model predictions, we created a dashboard in Tableau that would help any HR manager to retain the best talent by applying the correct strategies. This step-by-step post is just an example of what Advanced Analytics can do for your business, and of how easy it is to do it with the proper tools.

2.2.4 SCREENSHOT:

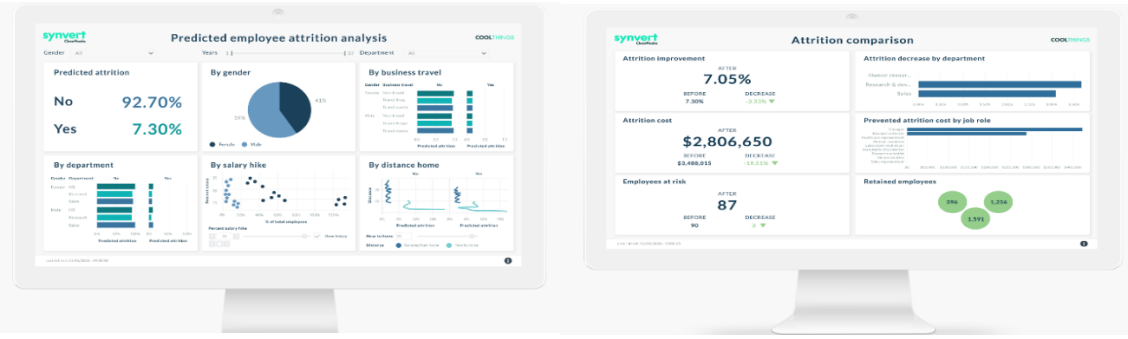


Figure 2.2.0, Figure 2.2.1

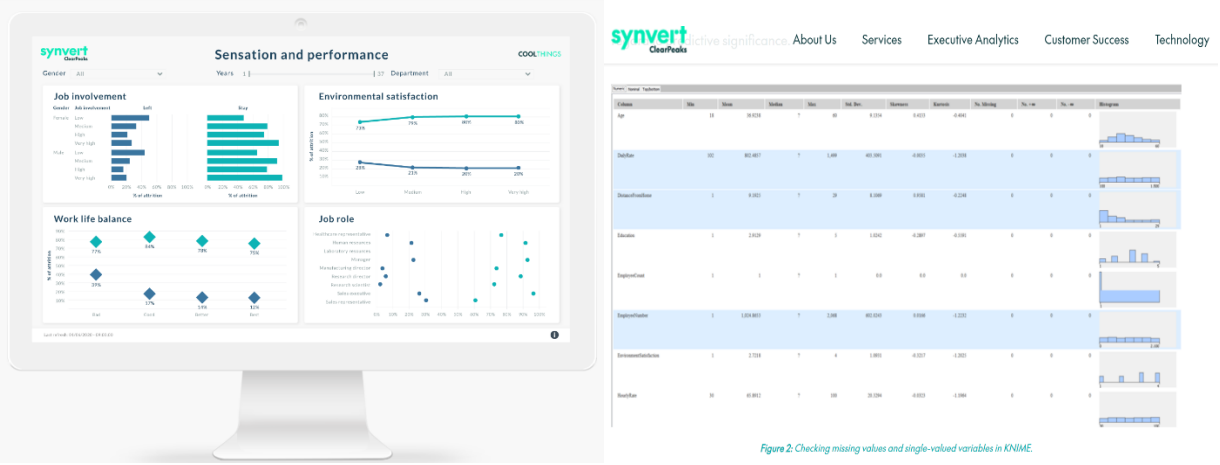


Figure 2.2.2, Figure 2.2.3

2.3 Hindawi

Automated Prediction of Employee Attrition Using Ensemble Model Based on Machine Learning Algorithms

2.3.1 INTRODUCTION:

Currently, machine learning and data mining are considered the most effective and active research areas. Different data mining techniques are used in classification, clustering, and prediction. Because of the importance of data mining and machine learning, many other methods are applied in different fields, such as education, healthcare, banking, security systems and human resource management. Employee attrition is a drop in the number of workers of an organization, where the employees have left the business voluntarily or retired. In any organization, highly efficient employees are considered the most valuable asset. Retaining the most marketable or high-performance employees is a big challenge in many organizations. In fact, this problem adds new spending on organizations to spend more on human capital, recruitment, preparation, and development for the new staff.

2.3.2 WORKING PRINCIPLE:

Employee attrition models implemented in many pieces of research. In their study, built a prediction model for employee churn rate. They used five machine learning algorithms, such as linear support vector machine and Decision tree and random forest outperformed all other classifiers generated different decision tree models and two rule sets. The datasets represent organizations of various sizes, ranging from small, medium, and large sized employee populations. The study concluded that no algorithm outperforms the others in the small dataset. In the medium dataset, the extreme gradient boosting trees result in greater accuracy, A prediction model for prioritizing the features with a high impact on employee attrition and its causes is presented.

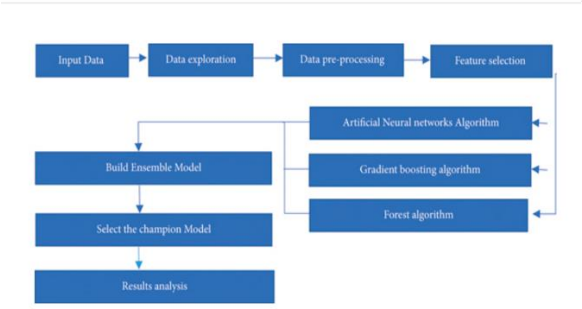
i. Proposed model
ii. Multilayer perceptron Classifier (MLP)

iii. Random Forest (RF)
iv. Gradient Boosting (GB)
v. Ensemble model
vi. Results

2.3.3 CONCLUSION:

The problem of maintaining good employees with experience threatens the owners of companies. The issue of employee attrition can cost employers a lot as it takes a lot to compensate for their expertise and efficiency. Hence, different machine learning techniques have been implemented with an ensemble model to find the different causes of such important business problems. In addition, different models fit statistic measures were proposed.

2.3.4 SCREENSHOT:



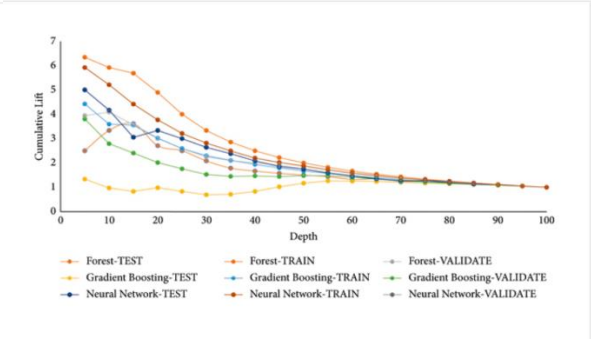
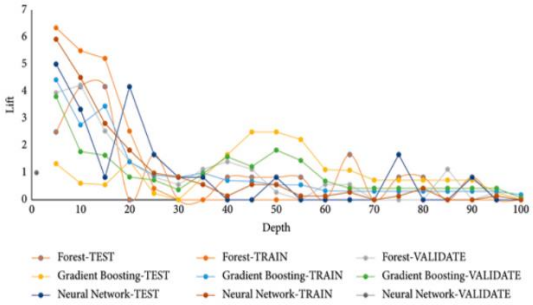


Figure 2.3.0, Figure 2.3.1



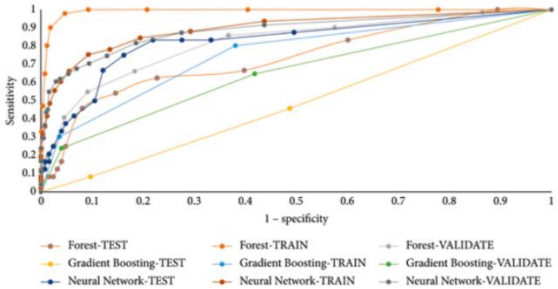
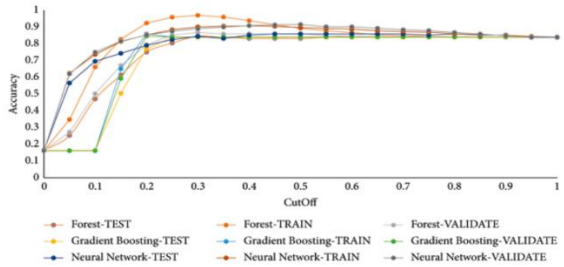


Figure 2.3.2, Figure 2.3.3



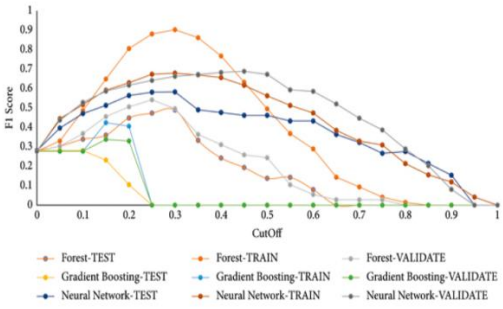


Figure 2.3.4, Figure 2.3.5

2.4 MDPI

Predicting Employee Attrition Using Machine Learning Approaches

2.4.1 INTRODUCTION:

Employee attrition is expressed as the normal process by which the employees leave the organization due to some reasons, such as the resignation of employees. There are many factors that can cause employee attrition. The employees leave the organization faster than they are hired. When the employee leaves the organization, the vacancies remain unfilled, resulting in a loss for the organization. The employee attrition rate helps to understand the progress level of an organization. The high attrition rate shows that the employees are frequently leaving. The results of the high attrition rate are the loss of organizational benefits. In order to keep the organization in progress, the attrition rate must be controlled. Many types of employee attrition help us to understand the attrition process. The attrition type is whether an employee chooses to leave the company voluntarily. The involuntary attrition type is when the organization ends the employment process. The external attrition type is referred to when an employee leaves an organization to work for another organization. Internal attrition occurs when an employee is given another position within the same organization as a promotion. The employee attrition rate is the measure of people who leaves the organization. By measuring the attrition rate, we can identify the causes and factors that need to be solved to eliminate employee attrition. The attrition rate is calculated by dividing the number of employees who have left the company by the average number of employees over some time. The attrition rate helps us find the company's progress over a specific period. The employee attrition states demonstrate that after six months of job duration, 1/3 of new employees leave the organization. The 3 to 4.5 million employees leave their job every month in the United States, according to the Job Openings and Labor Turnover Survey (JOLTS).

2.4.2 WORKING PRINCIPLE:

In the classification task, the Performance Assessment of Data Balancing Techniques was proposed. The imbalanced dataset causes a major issue in numerous classification problems such as intrusion detection, fraud detection, anomaly detection, and many more. The data balancing was

applied to obtain the high accuracy results from models. The research study addresses the performance issues empirically using an imbalanced dataset. prediction Hybrid Sampling (HS), Synthetic Minority Over Sampling (SMOTE), Under Sampling (US), Random Over Sampling Examples (ROSE), Over Sampling (OS), and Clustering-Based Under Sampling (CBUS) balancing techniques were examined. The imbalance ratio (IR) was used as the performance factor. The research experimental results show that data balancing proves helpful in improving the applied classifiers' performance. The results also indicate that no significant performance difference was found in US, SMOTE, HS, OS, and CBUS balancing techniques.

2.4.3 CONCLUSION:

The employee attrition prediction by using the four advanced machine learning techniques ETC, SVM, LR, and DTC, were applied in comparison in this study. The applied machine learning techniques achieved accuracy scores of 87% by SVM technique, 72% by LR technique, and 83% by DTC technique. The proposed Extra Trees Classifier (ETC) achieved 93% accuracy, precision, recall, f1 score, and ROC accuracy scores. The data resampling was applied to balance the dataset. The approaches were validated with k-fold validation and with the past applied state-of-the-art studies. By using dataset 10-folds, the SVM technique achieved an 88% of accuracy score, the LR technique achieved a 74% of accuracy score, the DTC technique achieved an 84% of accuracy score, and the proposed achieved a 93% of accuracy score. The EEDA application revealed the key factors that cause employee attrition is the monthly income, hourly rate, job level, and age. Our research findings help organizations overcome employee attrition. The study limitations and in future direction, we will apply the deep learning techniques to predict the employee attrition. Moreover, we will enhance the dataset feature space to obtain more accurate results by using deep learning techniques.

2.4.4 SCREENSHOT:

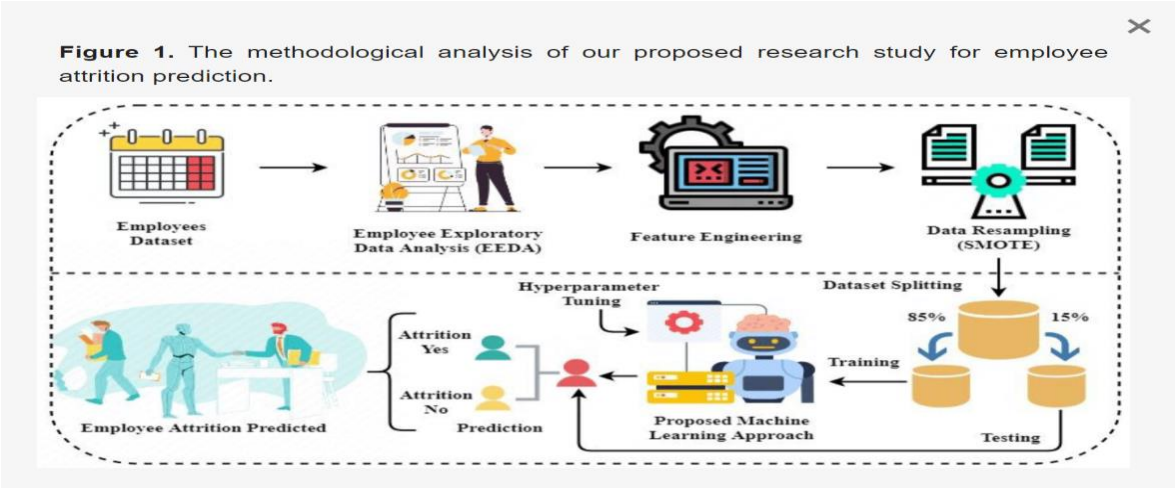


Figure 2.4.1

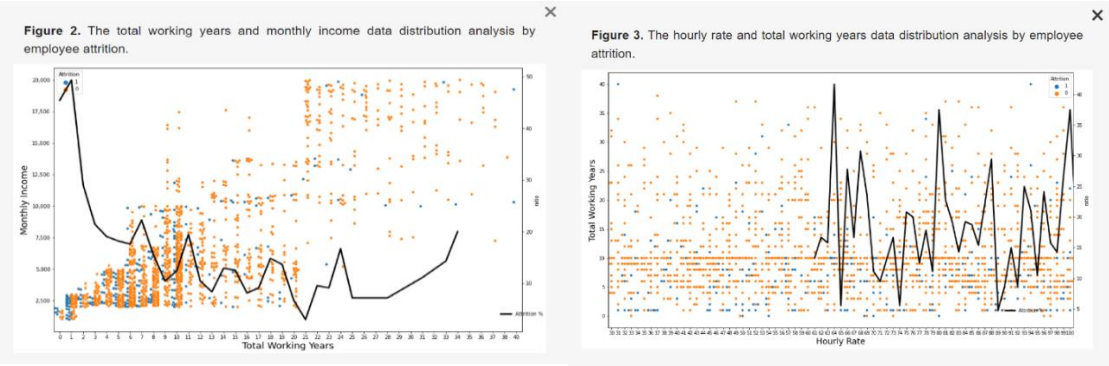


Figure 2.4.2, Figure 2.4.3

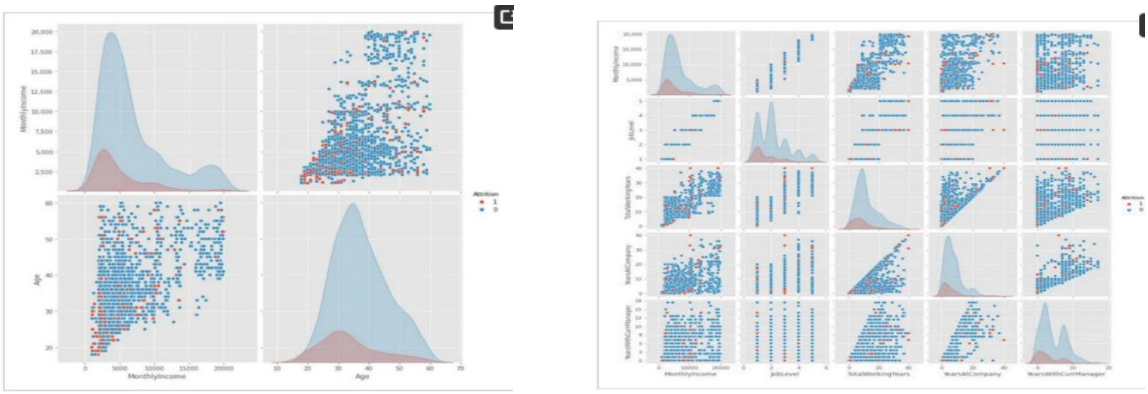


Figure 2.4.4, Figure 2.4.5

2.5 Global Scientific JOURNALS

Predicting Employee Attrition Using Decision Tree Algorithm

2.5.1 INTRODUCTION:

The growing interest in machine learning among business leaders and decision-makers demands that researchers explore its use within business organizations (Darrell and Allen, 2018). One of the major issues facing business leaders within companies is the loss of talented employees (Alduayj and Rajpoot, 2018). The key to success in an organization is the ability to attract and retain top talents (Rabbi, Ahad, Kousar, and Ali, 2015). The Human Resource (HR) department needs to identify the factors that keep employees and those which prompt them to leave (Frye, Boomhower, Smith, Vitovsky, and Fabricant, 2018). Organizations could do more to prevent the loss of good people. Employee attrition is the most important factor that causes a huge loss to the company (Chowdhury and Mamun, 2017). There are many reasons for which the employees leave the company, such as; salary, dissatisfaction, stagnant career growth, etc.

2.5.2 WORKING PRINCIPLE:

The research methodology adopted in this study is the Design Science Research (DSR). DSR is a problem-solving paradigm that seeks to enhance human knowledge via the creation of innovative artifacts. Simply stated, DSR seeks to enhance technology and science knowledge bases via the creation. Of innovative artifacts that solve problems and improve the environment in which they are instantiated.

2.5.3 CONCLUSION:

In conclusion, we successfully built our decision tree model using R and Transact SQL code and then stored the model in the SQL procedure. We then called the SQL procedure using C# to our web application to test it on new employee data. ASP.NET which comprises HTML, CSS, and JavaScript.

2.5.4 SCREENSHOT:

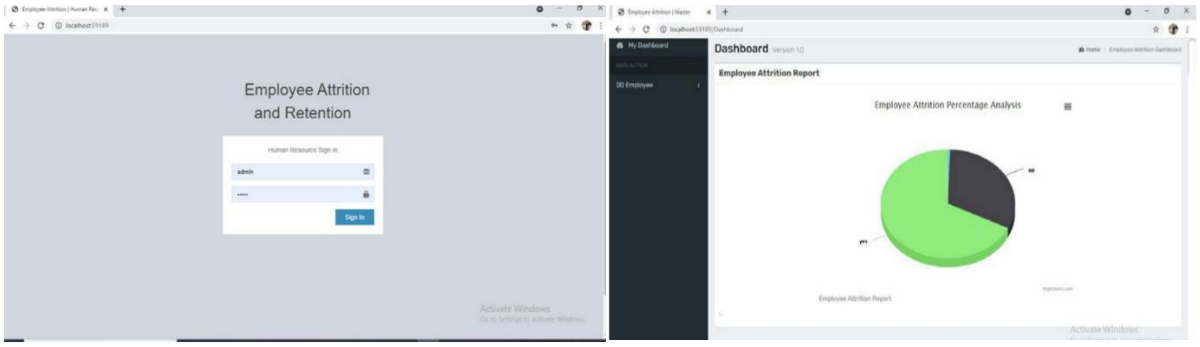


Figure 2.5.0, Figure 2.5.1

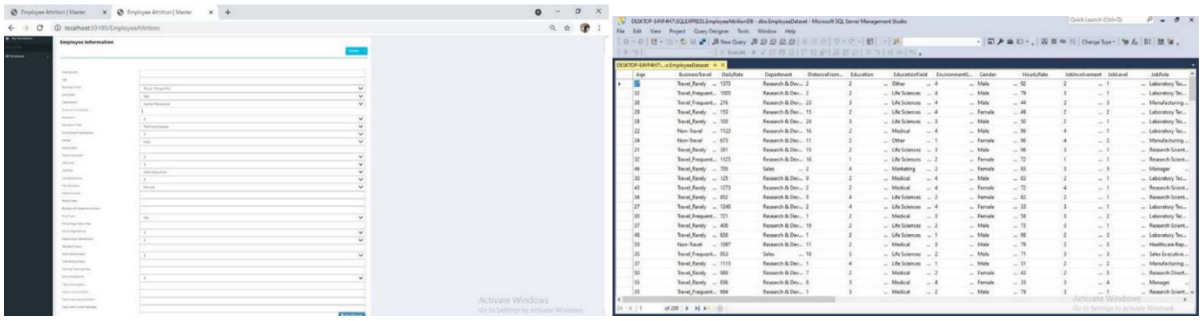


Figure 2.5.2, Figure 2.5.3

2.6 MAJOR DIFFERENCES BETWEEN OUR PROJECT AND OTHERS PROJECTS:

Others' Project Objectives	Your Project Objectives
1. Develop a Web Application Framework:	**1. Enhanced Machine Learning Models:**
Objective: To develop a user-friendly web application framework for hosting the attrition prediction system.	**Objective**: Develop and implement advanced machine learning models with enhanced accuracy and predictive capabilities for employee attrition.
Expected Result: A functional web interface accessible to users, providing a platform for interacting with the machine learning models.	**Expected Result:** Machine learning models that outperform existing benchmarks, providing more accurate and reliable predictions.
2. Implement Machine Learning Integration:	**2. Interactive Data Visualization Module:**
Objective: To integrate machine learning models into the web application for real-time predictions.	**Objective**: Integrate an interactive data visualization module within the web interface to provide users with dynamic insights into attrition patterns and contributing factors.
Expected Result: Seamless integration allowing users to input relevant data and receive attrition predictions through the web interface.	**Expected Result:** A visually appealing and user-friendly dashboard that allows users to explore and understand attrition data through charts, graphs, and interactive visualizations.
3. Design an Intuitive User Interface (UI):	**3. Personalized User Feedback Mechanism:**
Objective: To design an intuitive and visually appealing UI for the web application.	**Objective**: Implement a personalized user feedback mechanism to gather insights from HR professionals and organizational leaders regarding the effectiveness of attrition predictions.

Expected Result: A user interface that is easy to navigate, with clear input forms and visualizations of attrition predictions.	**Expected Result:** Continuous feedback loop enabling users to provide comments, suggestions, and real-world observations, fostering a collaborative improvement process.
4. Incorporate User Authentication and Authorization:	**4. Integration of Employee Engagement Metrics:**
Objective: To implement user authentication and authorization features for secure access to the web application.	**Objective**: Extend the system to incorporate employee engagement metrics, providing a holistic view of workplace satisfaction and its correlation with attrition.
Expected Result: A system that ensures only authorized personnel can access and utilize the attrition prediction functionalities.	**Expected Result:** The system not only predicts attrition but also correlates it with employee engagement metrics, helping organizations proactively address factors contributing to dissatisfaction.

Chapter # 03 Research Approach

3.1 Data Set

Predicting employee attrition is a common use case in HR analytics, aimed at identifying employees who are likely to leave the organization. Below is a sample structure for an employee attrition prediction dataset. This dataset typically includes various features related to employee demographics, job satisfaction, performance metrics, and other relevant factors.

3.1.2 Sample Dataset Structure

i. **Employee Information:**

   a. **EmployeeID**: Unique identifier for each employee.

   b. **Age**: Age of the employee.

   c. **Gender**: Gender of the employee.

   d. **MaritalStatus**: Marital status of the employee.

   e. **Education**: Education level of the employee.

   f. **EducationField**: Field of education.

   g. **DistanceFromHome**: Distance from home to workplace (in kilometers/miles).

ii. **Job Information:**

   a. **JobRole**: Role/designation of the employee.

   b. **Department**: Department of the employee.

   c. **JobLevel**: Level of the job (e.g., entry-level, mid-level, senior-level).

   d. **JobInvolvement**: Level of involvement in the job (e.g., on a scale from 1 to 5).

   e. **PerformanceRating**: Performance rating of the employee (e.g., on a scale from 1 to 5).

f. **YearsAtCompany**: Number of years the employee has been with the company.

g. **YearsInCurrentRole**: Number of years the employee has been in the current role.

h. **YearsSinceLastPromotion**: Number of years since the employee's last promotion.

i. **YearsWithCurrManager**: Number of years the employee has worked with the current manager.

iii. **Compensation and Benefits:**

a. **MonthlyIncome**: Monthly income of the employee.

b. **PercentSalaryHike**: Percent salary hike for the employee in the last appraisal.

c. **StockOptionLevel**: Level of stock options granted to the employee.

iv. **Work Environment:**

a. **WorkLifeBalance**: Work-life balance rating (e.g., on a scale from 1 to 5).

b. **JobSatisfaction**: Job satisfaction level (e.g., on a scale from 1 to 5).

c. **EnvironmentSatisfaction**: Satisfaction with the work environment (e.g., on a scale from 1 to 5).

d. **RelationshipSatisfaction**: Satisfaction with relationships at work (e.g., on a scale from 1 to 5).

v. **Behavioral Metrics:**

a. **OverTime**: Whether the employee works overtime (Yes/No).

b. **BusinessTravel**: Frequency of business travel (e.g., None, Travel_Rarely, Travel_Frequently).

vi. **Attrition:**

Attrition: Whether the employee has left the company (Yes/No).

3.2 Dataset Sources

For real-world datasets, you can refer to publicly available datasets such as the IBM HR Analytics Employee Attrition & Performance dataset, which can be found on platforms like Kaggle.

3.3 Algorithms Parameter Settings

The process of setting algorithm parameters, also known as hyper parameter tuning, is crucial for optimizing the performance of machine learning models. Here are common machine learning algorithms used for employee attrition prediction along with their key parameters and tips for setting them:

3.3.1 Common Algorithms and Their Parameters

i. **Logistic Regression**

- **C**: Inverse of regularization strength. Smaller values specify stronger regularization.
- Typical values: **0.01**, **0.1**, **1**, **10**, **100**
- **solver**: Algorithm to use in the optimization problem.
- Options: **liblinear**, **newton-cg**, **lbfgs**, **sag**, **saga**

ii. **Decision Trees**

- **max_depth**: Maximum depth of the tree.
- Typical values: **None**, **10**, **20**, **30**
- **min_samples_split**: Minimum number of samples required to split an internal node.
- Typical values: **2**, **10**, **20**
- **min_samples_leaf**: Minimum number of samples required to be at a leaf node.
- Typical values: **1**, **5**, **10**
- **criterion**: Function to measure the quality of a split.
- Options: **gini**, **entropy**

iii. **Random Forest**

- **n_estimators**: Number of trees in the forest.

- Typical values: **100**, **200**, **500**
- **max_features**: Number of features to consider when looking for the best split.
- Options: **auto**, **sqrt**, **log2**
- **max_depth**: Maximum depth of the tree.
- Typical values: **None**, **10**, **20**, **30**
- **min_samples_split**: Minimum number of samples required to split an internal node.
- Typical values: **2**, **10**, **20**
- **min_samples_leaf**: Minimum number of samples required to be at a leaf node.
- Typical values: **1**, **5**, **10**

iv. **Gradient Boosting Machines (GBM)**

- **n_estimators**: Number of boosting stages to be run.
- Typical values: **100**, **200**, **500**
- **learning_rate**: Learning rate shrinks the contribution of each tree.
- Typical values: **0.01**, **0.1**, **0.2**
- **max_depth**: Maximum depth of the individual regression estimators.
- Typical values: **3**, **5**, **7**
- **min_samples_split**: Minimum number of samples required to split an internal node.
- Typical values: **2**, **10**, **20**
- **min_samples_leaf**: Minimum number of samples required to be at a leaf node.
- Typical values: **1**, **5**, **10**

v. **Support Vector Machines (SVM)**

- **C**: Regularization parameter. The strength of the regularization is inversely proportional to C.
- Typical values: **0.1**, **1**, **10**, **100**
- **kernel**: Specifies the kernel type to be used in the algorithm.
- Options: **linear**, **poly**, **rbf**, **sigmoid**
- **gamma**: Kernel coefficient for **rbf**, **poly**, and **sigmoid**.
- Typical values: **scale**, **auto**, **0.1**, **1**, **10**

vi. Hyper parameter Tuning Methods

a. **Grid Search**:

- Exhaustively searches through a specified subset of hyper parameters.
- Suitable for smaller hyper parameter spaces.

b. **Random Search**:

- Randomly samples the hyper parameter space.
- More efficient than grid search for large spaces.

c. Bayesian Optimization:

- Uses Bayesian methods to find the optimum hyper parameters.
- Efficient for high-dimensional spaces.
- Libraries like hyperopt, optuna, or Bayesian Optimization can be used for this purpose.

3.4 Milestone Chart

S. No	Elapsed time since start of the project	Milestone	Deliverable
1.	Week 01	Timeline and GitHub Repository	Complete Timeline and GitHub Repository
2.	Week 02	Data Gathering & Environment Setup	Complete Data Gathering & Environment Setup
3.	Week 03	Login, Signup, Page Frontend and Backend	Complete Login, Signup, Page Frontend and Backend
4.	Week 04	Data Cleaning and Data Splitting	Complete Data Cleaning and Data Splitting
5.	Week 05	Model Training	Complete Model Training
6.	Week 06	Employees Description Frontend and Backend	Complete Employees Description Frontend and Backend
7.	Week 07	Employees Data Input Frontend and Backend	Complete Employees Data Input Frontend and Backend
8.	Week 08	Prediction Page Frontend and Backend	Complete Prediction Page Frontend and Backend
9.	Week 09	Database Tables Development	Complete Database Tables Development
10.	Week 10	Connection of Database	Complete Connection of Database
11	Week 11	APIs Development and Testing	Complete APIs Development and Testing
12	Week 12	Machine Learning Model Integration	Complete Machine Learning Model Integration
13	Week 13	Frontend and Backend API Integration	Complete Frontend and Backend API Integration
14	Week 14	Project Debugging and Testing	Complete Project Debugging and Testing

Chapter # 04 Requirement Analysis

4.1 Functional Requirements:

DATA COLLECTION AND PROCESSING

Consider specifying the types of data sources (e.g., performance metrics, satisfaction surveys) and the frequency of data collection.

Ensure compliance with data privacy regulations (e.g., GDPR, HIPAA) to protect employee information.

FEATURE SELECTION

Clarify the specific algorithms used for feature selection and the criteria for selecting relevant features.

MODEL DEVELOPMENT AND TRAINING

Provide details on the algorithms used for machine learning models and their rationale.

Specify the training schedule and conditions triggering retraining to adapt to evolving patterns.

PREDICTIVE ACCURACY

Define the acceptable level of accuracy and how it will be measured.

USER INTERFACE

Consider incorporating visualization tools to enhance the interpretability of predictions and insights.

INTEGRATION

Detail the integration process with existing HR systems, ensuring compatibility and minimal disruption.

4.2 Nonfunctional Requirements:

SCALABILITY

Specify scalability parameters (e.g., number of employees, data volume) and how the system will handle growth.

PERFORMANCE

Define specific response time requirements for real time or near real time insights.

RELIABILITY

Consider implementing failover mechanisms and backup systems to ensure high availability.

SECURITY

Detail encryption protocols, access controls, and auditing mechanisms to secure employee data.

COMPLIANCE

Clearly outline how the system ensures compliance with data protection regulations and ethical standards.

USABILITY

Define usability metrics and conduct user testing to validate the system's ease of use.

MAINTAINABILITY

Specify how updates, maintenance, and troubleshooting will be handled to minimize downtime.

INTEROPERABILITY

Enumerate specific HR software and tools that Talent Saver will integrate with.

4.3 USE CASE DIAGRAM

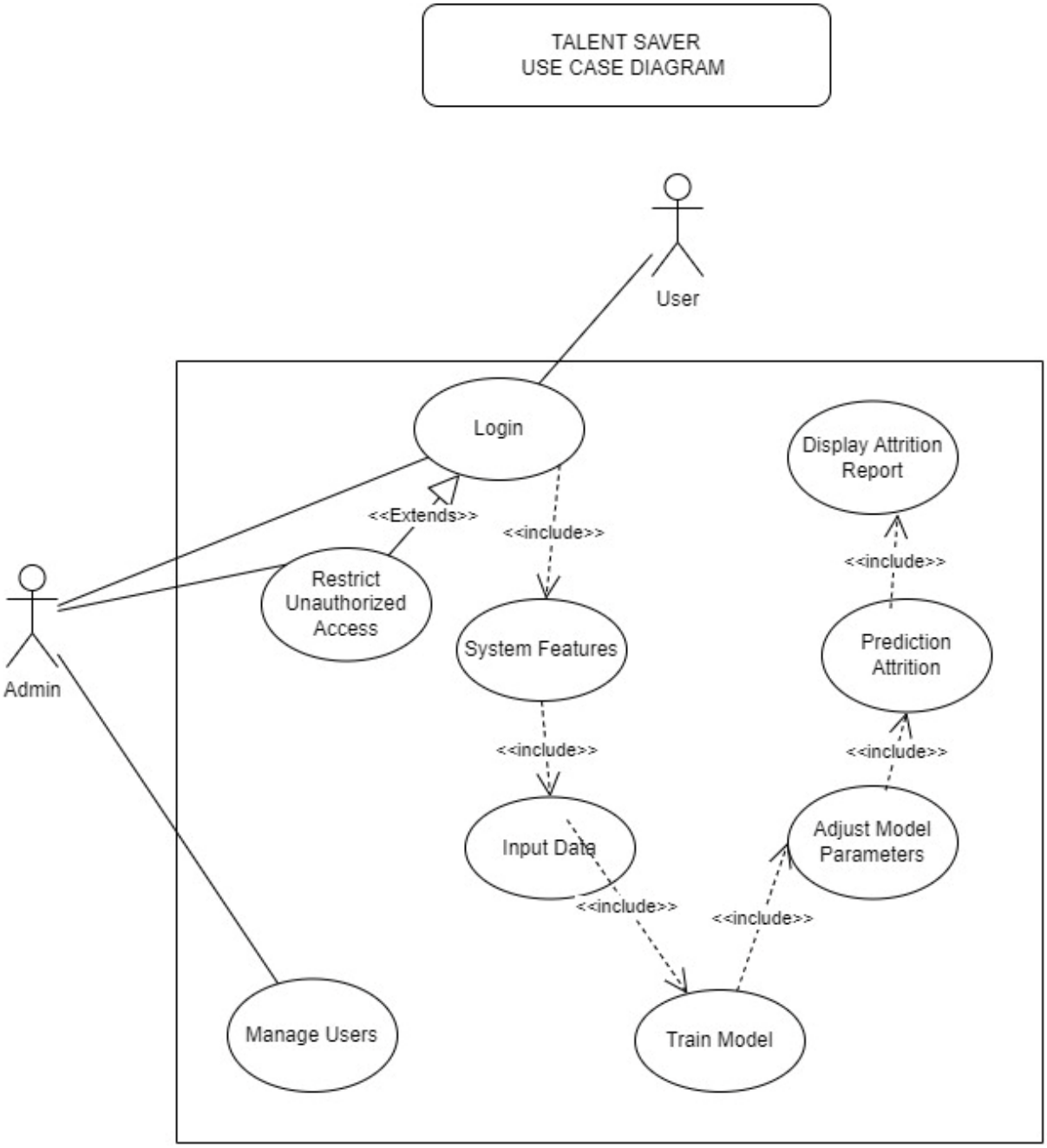


Figure 4.1

4.3.1 USE CASE DESCRIPTION

User Authentication	
Use Case:	Login
Description:	The HR Manager or Admin logs into the system.
Include Relationship:	The "Login" use case includes the "Access System Features" use case, meaning that successful login is a prerequisite for accessing system features.
Exclude Relationship:	If the login is unsuccessful or unauthorized, it excludes the "Restrict Unauthorized Access" use case, meaning that access to system features is restricted.
Model Training	
Use Case:	Train Model
Description:	The HR Manager initiates the training of the attrition prediction model after logging in.
Adjust Model Parameters	
Use Case:	Adjust Model Parameters
Description:	After training the model, the HR Manager may adjust parameters for fine-tuning.
Data Input	
Use Case:	Input Data
Description:	The HR Manager provides historical employee data for training the model.
Prediction	
Use Case:	Predict Attrition
Description:	The system predicts employee attrition based on the trained model.
Display Report	
Use Case:	Display Attrition Report
Description:	The system generates a report summarizing attrition predictions.
User Management (Admin)	
Use Case:	Manage Users
Description:	The Admin manages user-related functionalities.

4.4 UML CLASS DIAGRAM

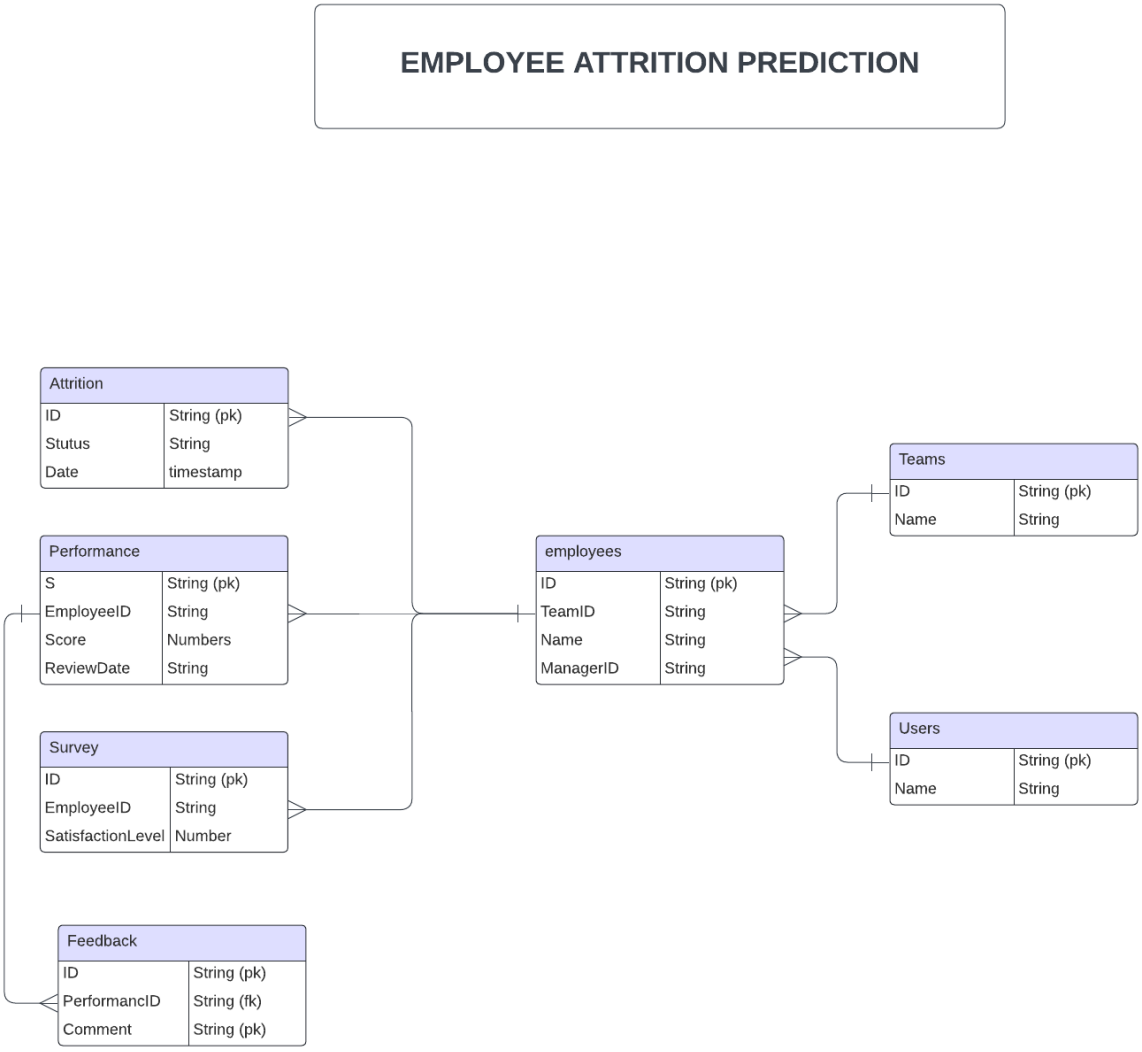


Figure 4.2

4.5 HIGHLY LEVEL ARCHITECTURE

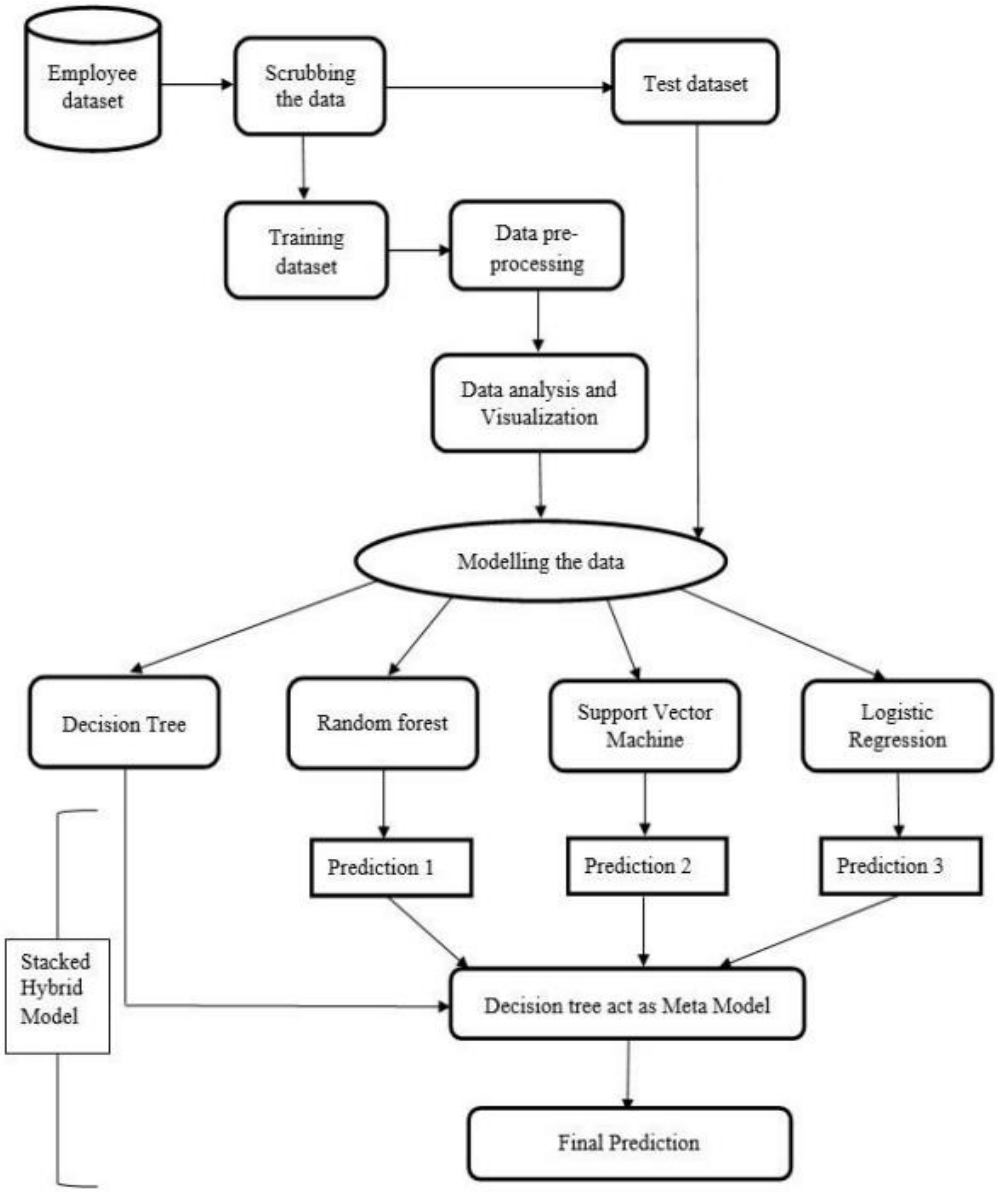


Figure 4.3

Chapter # 05 Design and Architecture

5.1 Architectural design

5.1.1 Class Diagram

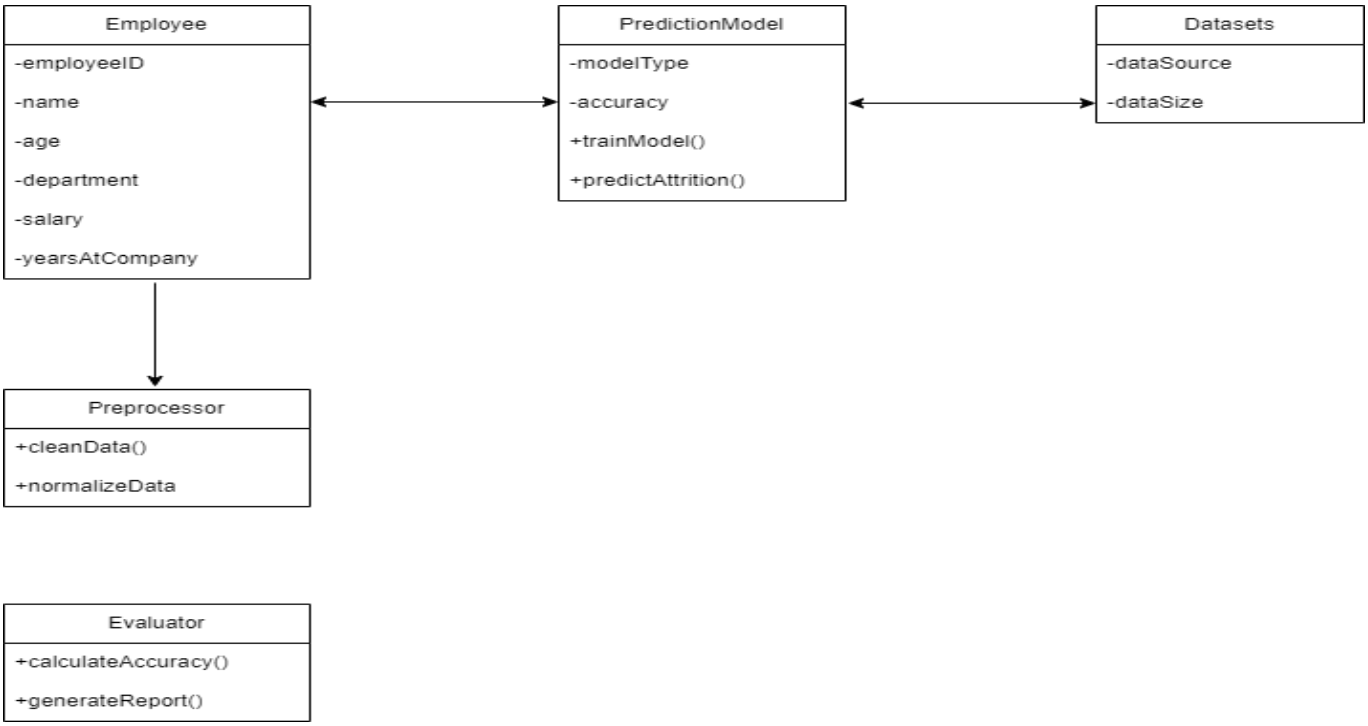
A class diagram provides a static view of the system, showcasing the different classes involved, their attributes, methods, and relationships.

Description:

- **Employee**: Represents an employee with attributes like employeeID, name, age, department, salary, yearsAtCompany, etc.
- **PredictionModel**: Represents the machine learning model used for predicting attrition with attributes like modelType, accuracy, and methods like trainModel(), predictAttrition().
- **Dataset**: Represents the dataset used for training and testing the model with attributes like dataSource, dataSize.
- **Preprocessor**: Handles data preprocessing tasks with methods like cleanData(), normalizeData().
- **Evaluator**: Evaluates the model's performance with methods like calculateAccuracy(), generateReport().

5.1.2 Class Diagram

Figure 5.1



5.1.3 State Transition Diagram

A state transition diagram shows the different states of the system and transitions between these states.

Description:

- **Idle**: Initial state where the system is awaiting input.
- **Preprocessing**: Data cleaning and normalization state.
- **Training**: State where the machine learning model is being trained.
- **Predicting**: State where the model makes predictions.
- **Evaluating**: State where the model's performance is evaluated.
- **Completed**: Final state after prediction and evaluation.

5.1.4 State Transition Diagram

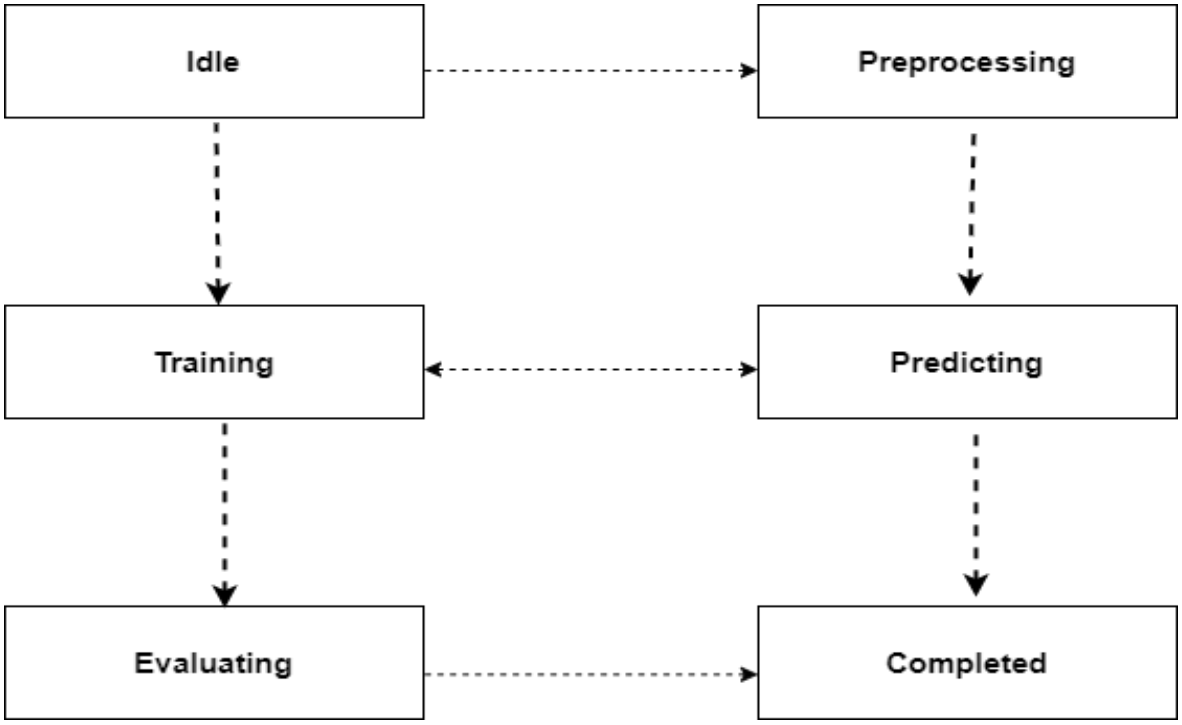


Figure 5.2

5.1.5 Data Flow Diagram (DFD)

A data flow diagram provides a high-level overview of the data flow within the system.

Description:

- **User**: Interacts with the system by inputting data and receiving predictions.
- **System**: Processes the user input through various subsystems (Preprocessor, PredictionModel, Evaluator).
- **Preprocessor**: Cleans and normalizes the input data.
- **PredictionModel**: Trains the model and makes predictions.
- **Evaluator**: Evaluates the model's performance and provides feedback.

5.1.6 Data Flow Diagram (DFD):

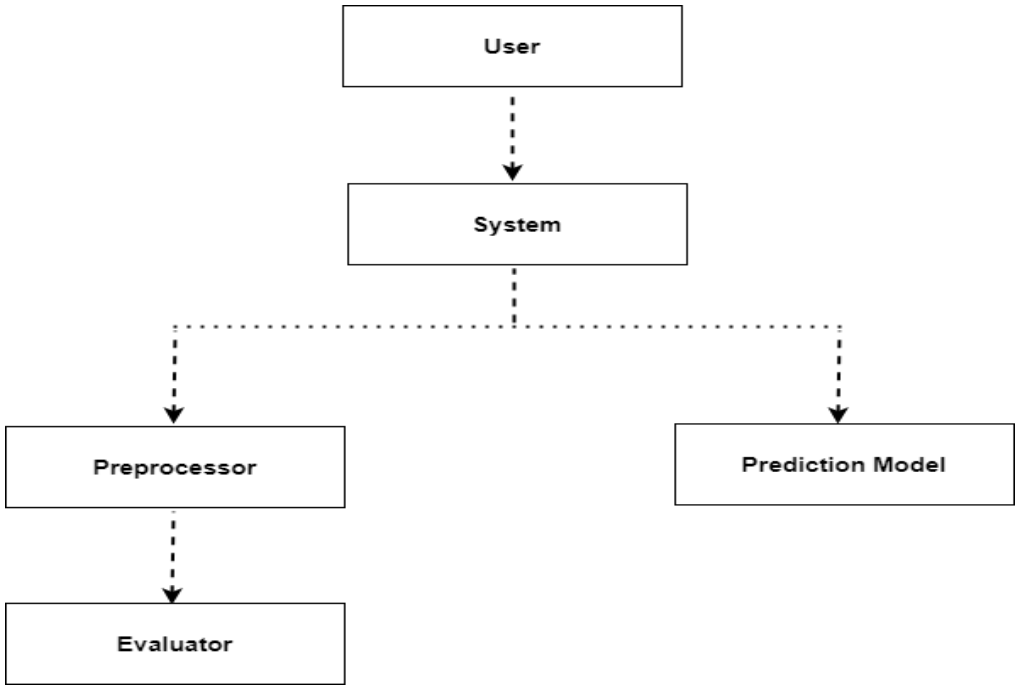


Figure 5.3

5.1.7 Unique Framework for Employee Attrition Prediction

This framework, named Enhanced Attrition Prediction Framework (EAPF), introduces a novel approach by integrating a hybrid ensemble method combining traditional machine learning models with advanced deep learning techniques, alongside incorporating real-time employee sentiment analysis from multiple data sources (emails, surveys, social media, etc.). The unique contribution lies in the combination of sentiment analysis and a hybrid ensemble method, which improves the prediction accuracy and provides deeper insights into the factors influencing employee attrition.

5.1.8 Framework Components:

i. Data Collection and Integration

ii. Data Preprocessing and Feature Engineering

iii. Sentiment Analysis Integration

iv. Hybrid Ensemble Model

v. Model Training and Validation

vi. Prediction and Interpretation

vii. Feedback Loop for Continuous Improvement

1. Data Collection and Integration

Gather data from various sources, including:

- **Employee Records**: HR databases, employee profiles, performance reviews.
- **Surveys and Feedback**: Employee satisfaction surveys, exit interviews.
- **Communication Data**: Emails, internal chat logs.
- **Social Media**: Public posts on platforms like LinkedIn, Twitter.
- **Algorithm Steps:**
    i. Connect to data sources and extract relevant data.
    ii. Merge datasets based on unique employee identifiers.
    iii. Ensure data integrity and consistency.

2. Data Preprocessing and Feature Engineering

Clean and prepare the data for analysis, extracting meaningful features.

Algorithm Steps:

i. **Data Cleaning**: Handle missing values, outliers, and duplicates.
ii. **Feature Extraction**: Extract features such as tenure, age, job satisfaction, etc.
iii. **Feature Transformation**: Normalize numerical features, encode categorical variables.
iv. **Feature Selection**: Use techniques like PCA or mutual information to select relevant features.

3. Sentiment Analysis Integration

Analyze sentiment from communication data to gauge employee morale and satisfaction.

Algorithm Steps:

i. Extract text data from communication sources.

ii. Preprocess text data (tokenization, stop-word removal, stemming).

iii. Apply sentiment analysis models (e.g., BERT, Vader) to derive sentiment scores.

iv. Integrate sentiment scores as features in the dataset.

4. Hybrid Ensemble Model

Develop a hybrid ensemble model combining traditional machine learning with deep learning methods.

Algorithm Steps:

i. **Traditional Models**: Train models like Decision Trees, Random Forest, and Logistic Regression.
ii. **Deep Learning Models**: Train models like LSTM or Transformer-based networks.
iii. **Ensemble Strategy**: Combine predictions using stacking, where the base models' predictions are fed into a meta-model for final prediction.

5. Model Training and Validation

Train the hybrid ensemble model and validate its performance.

Algorithm Steps:

i. Split the dataset into training, validation, and test sets.
ii. Train individual models on the training set.
iii. Use cross-validation to fine-tune hyper parameters.
iv. Train the ensemble model on base model predictions.
v. Validate the model on the validation set and assess performance using metrics like accuracy, precision, recall, and F1-score.

6. Prediction and Interpretation

Generate predictions and interpret the results to understand key factors influencing attrition.

Algorithm Steps:

i. Apply the trained model to the test set to predict attrition.

ii. Use interpretability techniques (e.g., SHAP values) to identify important features and their impact on predictions.

iii. Generate a report with actionable insights for HR and management.

7. Feedback Loop for Continuous Improvement

Incorporate feedback and new data to continuously improve the model.

Algorithm Steps:

i. Collect new data periodically and retrain the model.

ii. Monitor model performance over time and adjust as necessary.

iii. Incorporate feedback from HR and employees to refine features and models.

5.1.9 Flowchart of Enhanced Attrition Prediction Framework (EAPF):

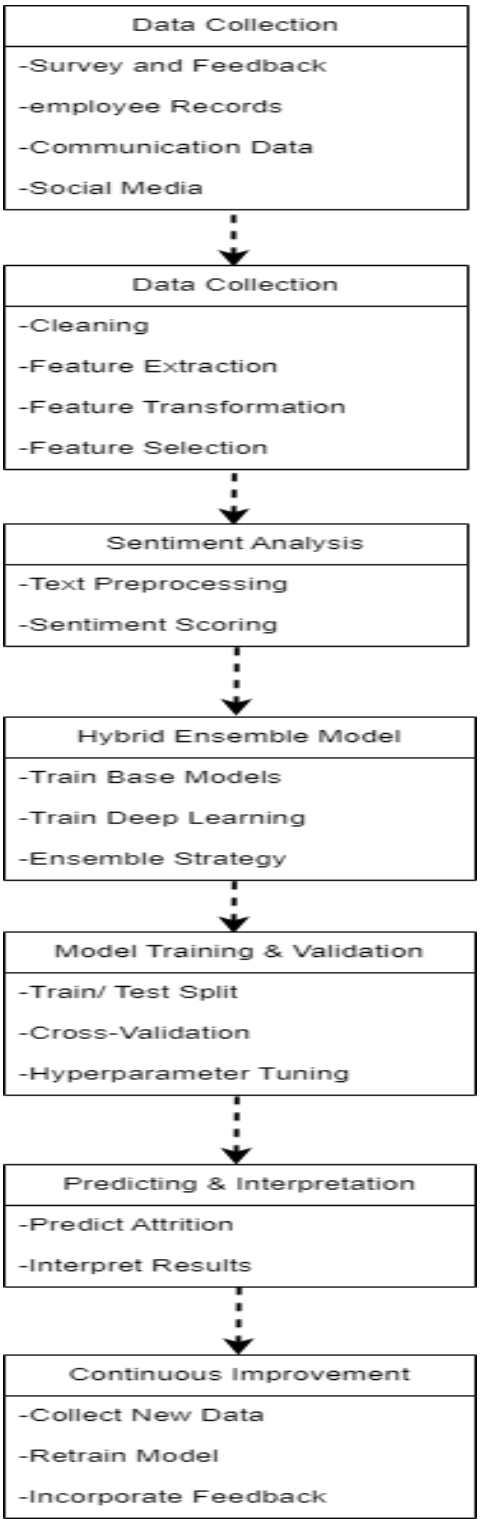


Figure 5.4

5.2 Data Design

Predicting employee attrition involves a series of methodologies that span data collection, processing, model building, evaluation, and deployment. Here are the detailed steps and methodologies typically used in an employee attrition prediction project.

Data Collection

- **Surveys**: Conducting employee satisfaction surveys.
- **HR Databases**: Extracting data from existing HR management systems.
- **Performance Records**: Collecting data on employee performance reviews and feedback.
- **External Sources**: Including industry benchmarks and economic data.

Data Preprocessing

- **Data Cleaning**: Handling missing values (e.g., imputation, removal), Correcting inaccuracies and inconsistencies.
- **Data Integration**: Merging data from different sources, Ensuring consistency across different datasets.
- **Data Transformation**: Encoding categorical variables (e.g., one-hot encoding), Scaling numerical features (e.g., standardization, normalization).
- **Feature Engineering**: Creating new features (e.g., tenure, engagement scores), Selecting important features (e.g., using correlation analysis, feature importance).

Exploratory Data Analysis (EDA)

- **Descriptive Statistics**: Calculating mean, median, mode, standard deviation, etc.
- **Visualization**: Plotting histograms, bar charts, and box plots, Using scatter plots and pair plots for multivariate analysis.
- **Correlation Analysis**: Heat maps to understand relationships between features.

Model Building

- **Supervised Learning Algorithms**:
- **Logistic Regression**: For binary classification tasks.

- **Decision Trees**: For interpretability and handling non-linear relationships.
- **Random Forest**: For reducing overfitting and improving accuracy.
- **Gradient Boosting Machines (e.g., XGBoost, LightGBM)**: For high performance on tabular data.
- **Support Vector Machines (SVM)**: For classification tasks with a clear margin of separation.
- **Neural Networks**: For complex patterns and relationships in data.

Model Training and hyper parameter tuning

- **Train-Test Split**: Dividing data into training and testing sets.
- **Cross-Validation**: Using k-fold cross-validation to ensure the model generalizes well.
- **Hyper parameter Tuning**:
- **Grid Search**: Exhaustive search over specified parameter values.
- **Random Search**: Randomly searching parameters for a specified number of iterations.
- **Bayesian Optimization**: Using probabilistic models to select hyper parameters efficiently.

Model Evaluation

- **Accuracy**: Overall correctness of the model.
- **Precision and Recall**: Precision measures the correctness of positive predictions, while recall measures the model's ability to find all positive instances.
- **F1 Score**: Harmonic mean of precision and recall.
- **ROC-AUC**: Area under the Receiver Operating Characteristic curve, measuring the trade-off between true positive rate and false positive rate.
- **Confusion Matrix**: Detailed breakdown of true positives, false positives, true negatives, and false negatives.

Model Deployment

- **API Development**: Creating RESTful APIs using Flask, FastAPI, or Django for serving predictions.

- **Containerization**: Using Docker to package the model and its dependencies.
- **Cloud Deployment**: Deploying on cloud platforms like AWS SageMaker, Google AI Platform, or Azure ML for scalability and maintenance.

Monitoring and Maintenance

- **Performance Monitoring**: Tracking model performance over time to detect degradation.
- **Drift Detection**: Identifying shifts in data distribution that affect model performance.

Feedback Loop

- **Continuous Improvement**: Incorporating feedback from users and updating the model.
- **A/B Testing**: Comparing the performance of different models or model versions in production.

Example Workflow

i. **Data Collection**: Gather employee data from HR systems and surveys.
ii. **Data Preprocessing**: Clean the data, encode categorical variables, and engineer relevant features.
iii. **EDA**: Perform exploratory data analysis to understand the data distribution and relationships.
iv. **Model Building**: Select and train several machine learning models.
v. **Model Evaluation**: Evaluate models using appropriate metrics and select the best-performing one.
vi. **Model Deployment**: Deploy the model using a Flask API and Docker.
vii. **Monitoring**: Continuously monitor the model's performance and set up alerts for any performance degradation.
viii. **Feedback Loop**: Regularly update the model with new data and feedback, retrain, and redeploy.

By following these methodologies, organizations can build robust employee attrition prediction systems that provide actionable insights to improve retention strategies.

5.3 Human interface design (product based or hardware system)

The TalentSaver application is designed to help users manage employee data, predict attrition, and find suitable replacements for leaving employees. Here's how users can interact with the system:

i. **Home Page**:

- **Description**: The home page provides an overview of the TalentSaver application, highlighting its purpose and key features.
- **User Interaction**: Users can navigate to other sections of the application using the navigation bar at the top, which includes links to "Home," "About," "Check Replacements," "Login," and "Logout."

ii. Signup Page:

- **Description**: New users can create an account by providing a username, email address, and password.
- **User Interaction**: Users fill in the required fields and click the "Signup" button to create their account. Once the account is created, they can log in using their credentials.

iii. Login Page:

- **Description**: Existing users can log in to their accounts by entering their email address and password.
- **User Interaction**: Users enter their credentials and click the "Login" button. Upon successful login, they are redirected to the main application interface.

iv. Check Replacements Page:

- **Description**: This page allows users to upload employee data in CSV format and find the best replacement for a leaving employee.
    - **User Interaction**:

- **Upload CSV File**: Users click the "Browse" button to select a CSV file containing employee data and then click "Upload."
- **Enter Employee ID**: Users enter the ID of the leaving employee in the provided text box.
- **Find Replacement**: Users click the "Find Replacement" button to initiate the process of finding the best replacement.

5.4 How the User Will Use the System to Complete All Expected Features

1. **Account Creation**:

    - Navigate to the "Signup" page.
    - Enter username, email, and password.
    - Click "Signup" to create an account.

2. **Logging In**:

    - Navigate to the "Login" page.
    - Enter email and password.
    - Click "Login" to access the main application.

3. **Uploading Employee Data**:

    - Navigate to the "Check Replacements" page.
    - Click "Browse" to select the CSV file containing employee data.
    - Click "Upload" to upload the file to the system.

4. **Finding a Replacement**:

    - Enter the employee ID of the leaving employee in the provided text box.
    - Click "Find Replacement" to process the data and display the best replacement candidate.

5.5 Feedback Information Displayed for the User

1. **Account Creation and Login**:

    - Confirmation messages indicating successful account creation or login.
    - Error messages if the account creation or login fails (e.g., incorrect password, email already in use).

2. **File Upload**:

    - Confirmation message indicating successful upload of the CSV file.
    - Error message if the file upload fails (e.g., invalid file format, upload error).

3. Replacement Finding:

    - Display of the best replacement candidate for the leaving employee, including relevant details such as name, current role, performance score, and other pertinent information.
    - Error message if the employee ID is not found or if there is an issue with processing the request.

By following these steps, users can efficiently utilize the TalentSaver application to manage employee data, predict attrition, and find suitable replacements for leaving employees. The system provides clear feedback at each stage to ensure a smooth user experience.

5.6 USER INTERFACE DESIGN – TEMPLATE

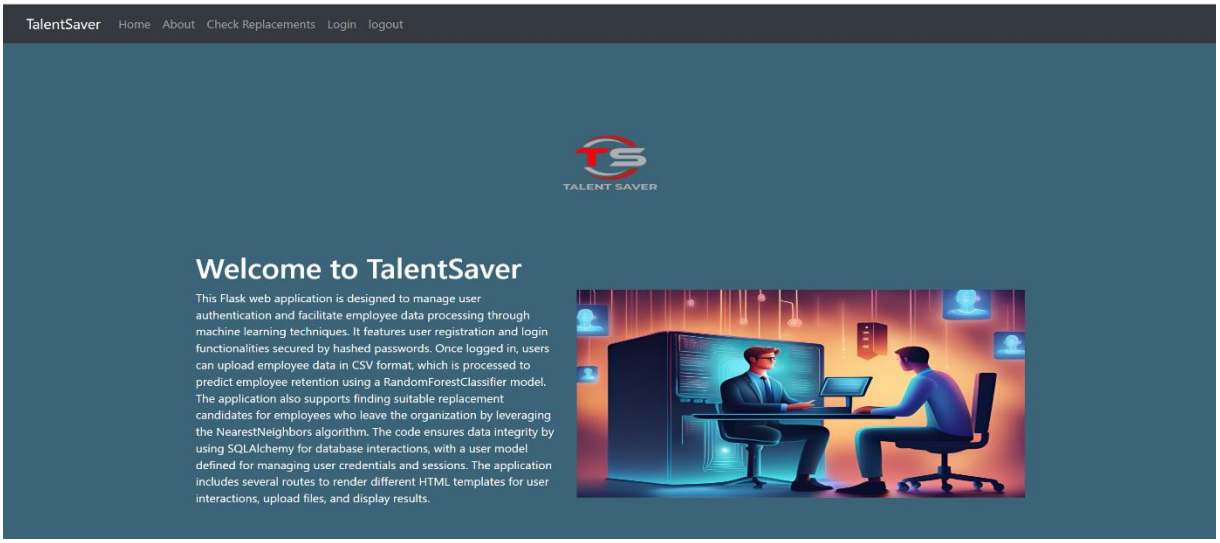
Project Title: TalentSaver **Date:** 25/06/2024 **Screen Name:** Home Page	
Screen: 1 of 5 **Link from screen:** None **Link to screen:** About, Check Replacements, Login, Logout	**Screen Description:** The Home Page provides an overview of the TalentSaver application, highlighting its purpose and key features. It welcomes users and guides them to the main functionalities of the system. The screen includes a navigation bar at the top and a section with a brief introduction and a related image.
Functionality/Interactivity: • Navigation bar links to other pages (About, Check Replacements, Login, and Logout). • Introduction text with a brief description of the application. • Image to visually represent the application.	
Screen Design:  	
Background: Gradient blue background	**Audio:** None
Color scheme: Blue gradient, white text	**Video:** None
Text attributes: Arial, 16pt, white	**Still images:** Illustration related to employee data and machine learning

Fig: 5.4.1

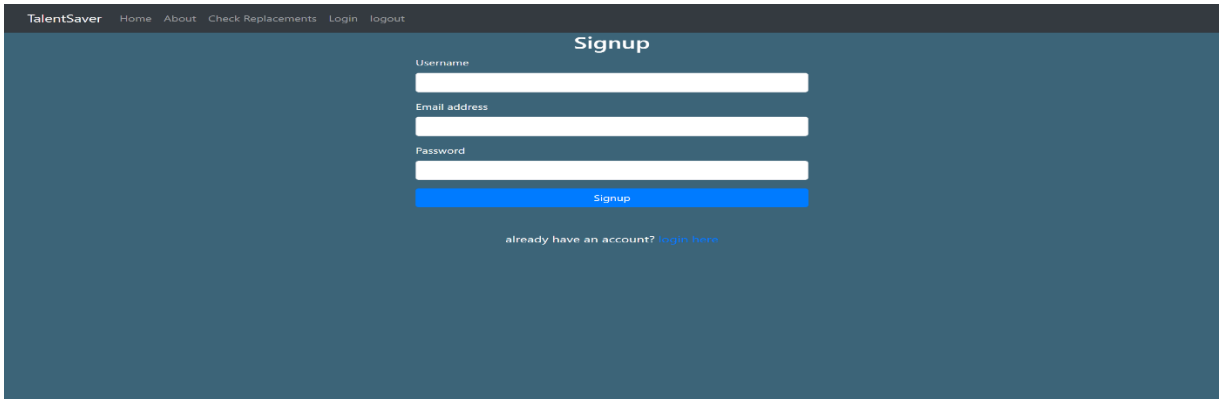
Project Title: TalentSaver **Date:** 25/06/2024 **Screen Name:** Signup Page	
Screen: 2 of 5 **Link from screen:** None **Link to screen:** Login	**Screen Description:** The Signup Page allows new users to create an account by providing a username, email address, and password. Once the account is created, users can log in using their credentials.
Functionality/Interactivity: • Input fields for username, email address, and password • Signup button to submit the form • Link to navigate to the Login page if the user already has an account	
Screen Design:  	
Background: Gradient blue background	**Audio:** None
Color scheme: Blue gradient, white text	**Video:** None
Text attributes: Arial, 16pt, white	**Still images:** None

Fig: 5.4.2

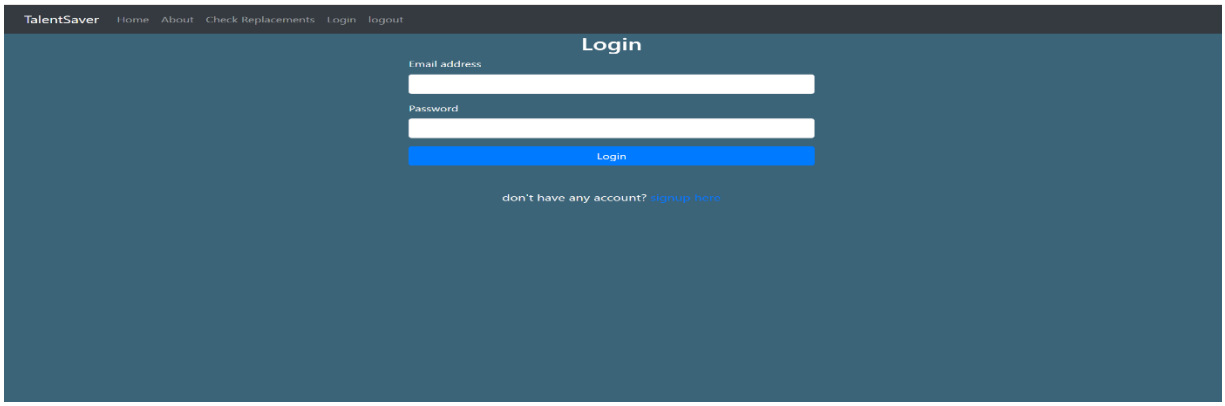
Project Title: TalentSaver **Date:** 25/06/2024 **Screen Name:** Login Page	
Screen: 3 of 5 **Link from screen:** None **Link to screen:** Check Replacements, Logout	**Screen Description:** The Login Page allows existing users to log in to their accounts by entering their email address and password.
Functionality/Interactivity: • Input fields for email address and password. • Login button to submit the form. • Link to navigate to the Signup page if the user does not have an account.	
Screen Design:  	
Background: Gradient blue background	**Audio:** None
Color scheme: Blue gradient, white text, blue button	**Video:** None
Text attributes: Arial, 16pt, white	**Still images:** None

Fig: 5.4.3

Project Title: TalentSaver
Date: 25/06/2024
Screen Name: Check Replacements Page

Screen: 4 of 5 **Link from screen:** None **Link to screen:** None	**Screen Description:** The Check Replacements Page allows users to upload employee data in CSV format and find the best replacement for a leaving employee by entering their Employee ID.

Functionality/Interactivity:

- File upload input to select and upload a CSV file.
- Text input for entering the Employee ID.
- Button to find the replacement for the specified employee.

Screen Design:

Background: Gradient blue background	**Audio:** None
Color scheme: Blue gradient, white text, grey button	**Video:** None
Text attributes: Arial, 16pt, white	**Still images:** None

Fig: 5.4.4

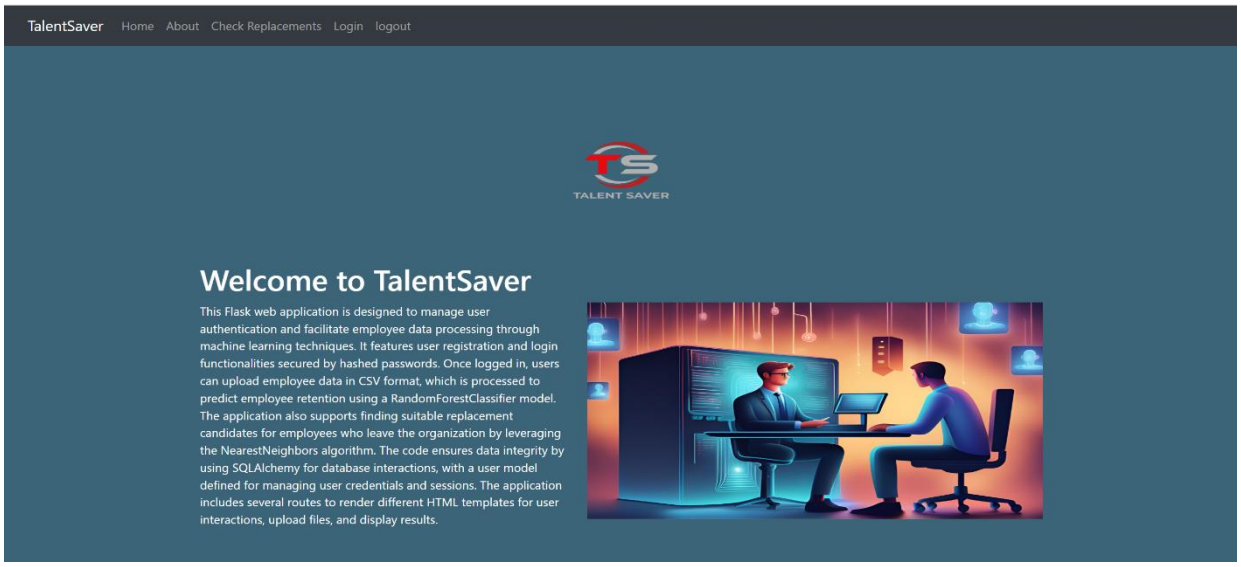
Project Title: TalentSaver **Date:** 25/06/2024 **Screen Name:** About Page	
Screen: 5 of 5 **Link from screen:** None **Link to screen:** Home	**Screen Description:** The About Page provides information about the TalentSaver application, its purpose, and its features.
Functionality/Interactivity: • Text content explaining the application • Link to navigate back to the Home page	
Screen Design:  	
Background: Gradient blue background	**Audio:** None
Color scheme: Blue gradient, white text	**Video:** None
Text attributes: Arial, 16pt, white	**Still images:** None

Fig: 5.4.5

This template provides a detailed description of the functionality, interactivity, and design of each screen in the TalentSaver application.

5.6.1 Usability Heuristics for User Interface Design

Project Title: **TalentSaver**

5.6.2 Clarity, Familiarity, User Control, and Flexibility

All screens in the TalentSaver application adhere to important rules of interface design: clarity, familiarity, user control, and flexibility.

- **Clarity**: The design ensures that all elements are easily understandable. The navigation and functions are straightforward, making it easy for users to find what they need without confusion.
- **Familiarity**: Standard interface elements such as text fields for username, email, and password are used to ensure familiarity. This helps users feel comfortable and reduces the learning curve.
- **User Control**: Users have control over their actions within the application. They can easily navigate between pages, upload files, and check replacements as needed.
- **Flexibility**: The application is designed to be responsive, working seamlessly on computers, tablets, and smartphones without compromising design or functionality.

Navigation in the screens is simple and intuitive. The flow of information follows a natural path – left to right and top to bottom. Navigation between screens is clear and uses familiar elements like buttons at the top of the screens. Inputs are required to minimize memory load and prevent errors, with error messages detailing any problems encountered.

5.7 Hierarchy and Negative Space

The application screens exhibit a good visual hierarchy. Information is organized based on importance, with sections clearly headed and/or framed to emphasize content and reduce cognitive load.

- **Hierarchy**: Titles and frames are used to break down information, making it easier to digest. Action buttons such as "Signup," "Login," "Upload," and "Find Replacement" are positioned where users expect them, next to input fields or at the bottom of the screen.
- **Negative Space**: The layout employs negative space effectively, ensuring that the interface is not cluttered. Text is justified to the left, following a natural reading flow from top to bottom and left to right. A simple font and a blue/white color scheme enhance familiarity and readability.

5.8 Accessibility

Accessibility is a key consideration in the design of TalentSaver, following guidelines set out by W3C Web Accessibility Initiative (2023) and Digital.Govt.NZ.

- **Visual Accessibility**: The content is easy to see, using a simple, readable font with high contrast between foreground and background colors. Color is not the sole means of conveying information or identifying content.
- **Scalability**: Screens can be resized without loss of content or functionality. The application remains clear and functional across different devices and screen sizes.
- **Cognitive Load**: The design minimizes cognitive load by using appropriate headings and everyday language. Fields are required to reduce user mistakes, and helpful error messages are provided.
- **Predictability**: Content is presented in a clear, coherent, and predictable manner, ensuring a smooth user experience.

5.9 Summary

The TalentSaver application prioritizes usability through clarity, familiarity, user control, flexibility, hierarchy, negative space, and accessibility. By adhering to these principles, the application ensures a user-friendly interface that is intuitive, accessible, and efficient.

Chapter # 06 Implementation

This chapter outlines the implementation details of the employee attrition prediction project, focusing on the algorithms used to build the predictive model. Several machine learning algorithms were utilized, each offering different strengths in handling the task of employee attrition prediction. Below, we discuss the major algorithms, their functioning, and pseudocode where applicable.

6.1 Algorithms

i. Logistic Regression
ii. Decision Tree Classifier
iii. Random Forest Classifier
iv. Gaussian Naive Bayes
v. K-Nearest Neighbors (KNN) Classifier
vi. Support Vector Machine (SVM)

1. Logistic Regression Purpose:

Logistic Regression is used for binary classification tasks, where the goal is to predict whether an employee will leave (attrition) or stay.

2. Decision Tree Classifier Purpose:

The Decision Tree algorithm is a non-parametric supervised learning method used for classification. It splits the data into branches to classify whether an employee will leave or stay.

3. Random Forest Classifier Purpose:

Random Forest is an ensemble learning method that constructs multiple decision trees during training and outputs the mode of the classes for classification tasks.

4. Gaussian Naive Bayes Purpose:

Gaussian Naive Bayes is a probabilistic classifier that applies Bayes' theorem with the assumption that the features follow a normal (Gaussian) distribution.

5. K-Nearest Neighbors (KNN) Classifier Purpose:

KNN is a simple, non-parametric method used for classification. It classifies a sample based on the majority class among its k-nearest neighbors.

6. Support Vector Machine (SVM) Purpose:

SVM is a supervised learning model used for binary classification. It finds the hyperplane that best separates the classes with the maximum margin.

6.2 Explanation of Main Features:

1. Data Preprocessing:

Feature Scaling: Most algorithms, particularly SVM and KNN, perform better with scaled data. Features are normalized or standardized to a common scale.

Handling Missing Values: Missing data are either imputed or removed to ensure the algorithms receive complete datasets.

2. Model Training:

Fitting the Model: Each algorithm is trained on a portion of the dataset (X_train, y_train) to learn the patterns and relationships between features and the target variable (attrition).

Hyper parameter Tuning: Parameters such as n_neighbors in KNN or n_estimators in Random Forest are tuned to optimize the model's performance.

3. Prediction:

Generating Predictions: After training, the model generates predictions (y_pred) on the test set (X_test), providing the likelihood of employee attrition.

4. Model Evaluation:

Performance Metrics: Models are evaluated using accuracy, precision, recall, F1-score, and ROC-AUC to determine their effectiveness in predicting attrition.

Cross-Validation: Cross-validation may be used to assess the model's stability and generalizability.

Chapter # 07 Testing and Evaluation

7.1 Testing Phase

Test Case Id: 1.1		**Pass/Fail:**	Pass
Test Type:	Functionality	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system correctly predicts employee attrition when provided with a valid employee ID.		
Test Steps:	1. Open the Talent Saver web application. 2. Enter a valid employee ID in the input field. 3. Click the "Predict Attrition" button. 4. Observe the prediction result.		
Expected Result:	The system should correctly display whether the employee will stay or leave and show a suitable replacement.		
Actual Result:	The system correctly displays whether the employee will stay or leave and shows a suitable replacement from the dataset.		
Date Prepared:	28/07/24.	**Date Run:**	03/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.2		**Pass/Fail:**	Pass
Test Type:	Validation	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system displays an error message when an invalid or non-existent employee ID is entered.		
Test Steps:	1. Enter an invalid employee ID. 2. Click the "Predict Attrition" button. 3. Observe the error message.		
Expected Result:	The system should display an error message indicating that the ID is invalid or not found.		
Actual Result:	The system displays an error message indicating that the employee ID is invalid or not found.		
Date Prepared:	28/07/24.	**Date Run:**	03/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.3		**Pass/Fail:**	Pass
Test Type:	Functionality	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system suggests the best replacement from the same department as the leaving employee.		
Test Steps:	1. Enter a valid employee ID of an employee predicted to leave. 2. Click the "Predict Attrition" button. 3. Check the suggested replacement's department.		
Expected Result:	The replacement should be from the same department as the leaving employee.		
Actual Result:	The replacement suggested by the system is from the same department as the leaving employee		
Date Prepared:	28/07/24.	**Date Run:**	03/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.4		**Pass/Fail:**	Pass
Test Type:	Functionality	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the replacement suggestion does not include employees who have already left.		
Test Steps:	1. Enter a valid employee ID of an employee predicted to leave. 2. Click the "Predict Attrition" button. 3. Observe the suggested replacement.		
Expected Result:	The replacement should only include current employees, not those who have left.		
Actual Result:	The system suggests only current employees as replacements, excluding those who have already left the company		
Date Prepared:	28/07/24.	**Date Run:**	03/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.5		**Pass/Fail:**	Pass
Test Type:	Usability	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the input field for employee ID only accepts numeric values.		
Test Steps:	1. Try entering alphabetic characters in the employee ID field. 2. Try entering special characters in the employee ID field. 3. Click the "Predict Attrition" button.		
Expected Result:	The input field should only accept numeric values and prompt an error for non-numeric input.		
Actual Result:	The input field for employee ID only accepts numeric values. If non-numeric characters are entered, the system prompts an error.		
Date Prepared:	28/07/24.	**Date Run:**	03/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.6		**Pass/Fail:**	Pass
Test Type:	Functionality	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system correctly identifies employees with a "High" attrition risk.		
Test Steps:	1. Enter an employee ID of an employee likely to leave. 2. Click the "Predict Attrition" button. 3. Observe the risk level displayed.		
Expected Result:	The system should display a "High" risk level for employees likely to leave.		
Actual Result:	The system correctly displays a "High" risk level for employees likely to leave the company		
Date Prepared:	04/07/24.	**Date Run:**	09/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.7		**Pass/Fail:**	Pass
Test Type:	Functionality	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the output table on the homepage displays employee details in the correct order: Employee ID, Name, Age, etc.		
Test Steps:	1. Enter a valid employee ID. 2. Click the "Predict Attrition" button. 3. Verify the order of columns in the output table.		
Expected Result:	The output table should display the columns in the order: Employee ID, Name, Age, etc.		
Actual Result:	The output table on the homepage displays employee details in the order: Employee ID, Name, Age, and so on.		
Date Prepared:	04/07/24.	**Date Run:**	09/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.8		**Pass/Fail:**	Pass
Test Type:	Performance	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system responds within 2 seconds after clicking the "Predict Attrition" button.		
Test Steps:	1. Enter a valid employee ID. 2. Click the "Predict Attrition" button. 3. Measure the response time.		
Expected Result:	The system should respond within 2 seconds.		
Actual Result:	The system responds within 2 seconds after clicking the "Predict Attrition" button.		
Date Prepared:	04/07/24.	**Date Run:**	09/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.9		**Pass/Fail:**	Pass
Test Type:	Security	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the application is secure against SQL injection attacks.		
Test Steps:	1. Enter a SQL injection string in the employee ID field. 2. Click the "Predict Attrition" button.		
Expected Result:	The system should handle the input securely without revealing any database information or executing malicious SQL commands.		
Actual Result:	The system securely handles the input without executing malicious SQL commands or revealing any database information.		
Date Prepared:	04/07/24.	**Date Run:**	09/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.10		**Pass/Fail:**	Pass
Test Type:	Validation	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system does not accept empty input for employee ID.		
Test Steps:	1. Leave the employee ID field empty. 2. Click the "Predict Attrition" button.		
Expected Result:	The system should prompt the user to enter an employee ID.		
Actual Result:	The system prompts the user to enter an employee ID if the input field is left empty.		
Date Prepared:	04/07/24.	**Date Run:**	09/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.11		**Pass/Fail:**	Pass
Test Type:	Functionality	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system correctly handles the case where multiple employees share the same name.		
Test Steps:	1. Enter an employee ID of an employee who shares a name with another employee. 2. Click the "Predict Attrition" button. 3. Observe the displayed details.		
Expected Result:	The system should correctly display details corresponding to the provided employee ID, even if multiple employees share the same name.		
Actual Result:	The system correctly displays details corresponding to the provided employee ID, even if multiple employees share the same name.		
Date Prepared:	11/08/24.	**Date Run:**	15/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.12		**Pass/Fail:**	Pass
Test Type:	Usability	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system allows users to easily navigate between the input form and the results table.		
Test Steps:	1. Enter a valid employee ID. 2. Click the "Predict Attrition" button. 3. Observe the ease of navigation between input and results.		
Expected Result:	The interface should be user-friendly, with clear navigation options between different sections.		
Actual Result:	The user interface allows easy navigation between the input form and the results table, making it user-friendly.		
Date Prepared:	11/08/24.	**Date Run:**	15/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.13		**Pass/Fail:**	Pass
Test Type:	Functionality	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system correctly identifies and suggests replacements for all employees marked as "Promoted."		
Test Steps:	1. Enter a valid employee ID of a promoted employee. 2. Click the "Predict Attrition" button. 3. Observe the replacement suggestion.		
Expected Result:	The system should correctly suggest a replacement for promoted employees.		
Actual Result:	The system correctly identifies and suggests replacements for all promoted employees.		
Date Prepared:	11/08/24.	**Date Run:**	15/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.14		**Pass/Fail:**	Pass
Test Type:	Compatibility	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system works correctly across different web browsers (Chrome, Firefox, Edge, etc.).		
Test Steps:	1. Open the Talent Saver web application in different browsers. 2. Enter a valid employee ID. 3. Click the "Predict Attrition" button.		
Expected Result:	The system should function correctly and consistently across different web browsers.		
Actual Result:	The system functions correctly and consistently across different web browsers.		
Date Prepared:	11/08/24.	**Date Run:**	15/08/24.
Tested By:	Talent Saver FYP members.		

Test Case Id: 1.15		**Pass/Fail:**	Pass
Test Type:	Data Integrity	**Prepared By:**	Shoaib Ahmed
Test Case Description:	Verify that the system maintains data integrity when the same employee ID is used in multiple sessions.		
Test Steps:	1. Enter a valid employee ID and predict attrition in one session. 2. Open a new session and enter the same employee ID. 3. Predict attrition again.		
Expected Result:	The system should maintain data integrity and produce consistent results across sessions.		
Actual Result:	The system maintains data integrity and produces consistent results when the same employee ID is used in multiple sessions.		
Date Prepared:	11/08/24.	**Date Run:**	15/08/24.
Tested By:	Talent Saver FYP members.		

Chapter # 08 Conclusion and Future Work

8.1 Conclusion

The TalentSaver project represents a significant step forward in addressing the persistent challenge of employee attrition within organizations. Leveraging advanced machine learning techniques, the system provides an innovative approach to predicting and mitigating employee turnover. Unlike traditional methods, TalentSaver integrates a comprehensive analysis of employee engagement metrics, offering a deeper understanding of the factors contributing to employee dissatisfaction. The user-friendly web application framework, coupled with real-time prediction capabilities, ensures that organizations can proactively address potential attrition issues, thereby reducing associated costs and maintaining productivity.

TalentSaver's emphasis on mobile responsiveness and continuous improvement through user feedback sets it apart from existing solutions. By providing actionable insights and personalized recommendations, the system not only helps in retaining valuable talent but also aids in creating a more positive and engaging workplace environment. As organizations continue to adapt to evolving workforce dynamics, TalentSaver emerges as a crucial tool in strategic human resource planning and talent management.

The successful implementation of TalentSaver has demonstrated its potential in reshaping talent management practices. By addressing the root causes of employee turnover, the system contributes to enhanced employee engagement, reduced recruitment costs, and a more stable workforce. As a holistic solution, Talent Saver is poised to make a lasting impact on the landscape of human resource management.

8.2 System Limitations and Challenges:

While the TalentSaver project offers numerous benefits, it also faces several limitations and challenges that need to be addressed to ensure its effectiveness and scalability.

i. Data Quality and Availability:

- The accuracy of the prediction models heavily relies on the quality and comprehensiveness of the input data. Inconsistent, incomplete, or biased data can lead to inaccurate predictions and potentially misguide HR decisions.
- Obtaining high-quality, up-to-date data from various sources such as HR systems, employee surveys, and performance evaluations can be challenging and time-consuming.

ii. Model Accuracy and Bias:

- Machine learning models may exhibit biases based on historical data, which could result in unfair or discriminatory predictions. Ensuring fairness and avoiding bias in the models is a critical challenge that requires continuous monitoring and adjustment.
- The accuracy of the models in predicting employee attrition may vary across different organizations and industries, necessitating customization and fine-tuning for specific contexts.

iii. Integration with Existing Systems:

- Integrating the TalentSaver system with existing HR systems and workflows can be complex and may require significant technical effort. Ensuring seamless data flow and compatibility with diverse HR platforms is essential for the system's adoption.
- User adoption and training: Ensuring that HR professionals and organizational leaders are adequately trained to use the TalentSaver system effectively is crucial. Resistance to change and a lack of technical expertise can hinder the system's successful implementation and utilization.
- **Privacy and Security Concerns:**
- Handling sensitive employee data necessitates robust data privacy and security measures. Ensuring compliance with data protection regulations such as GDPR is paramount to maintaining employee trust and safeguarding organizational information.
- Balancing data accessibility for predictive modeling with stringent privacy requirements presents an ongoing challenge.

v. Scalability and Performance:

- As organizations grow and the volume of employee data increases, ensuring the scalability and performance of the TalentSaver system becomes critical. The system must be capable of handling large datasets and providing real-time predictions without compromising on speed or accuracy.
- Continuous monitoring and optimization of the system's performance are necessary to maintain its effectiveness under varying usage scenarios.

8.3 Future Work

i. Enhanced Data Integration and Preprocessing:

- Future work should focus on improving data integration capabilities to streamline the process of collecting and preprocessing employee data from diverse sources. Developing automated data cleansing and normalization techniques can enhance data quality and consistency.
- Expanding data sources to include additional metrics such as employee engagement surveys, social media interactions, and external labor market trends can provide a more comprehensive view of employee satisfaction and attrition risks.

ii. Advanced Predictive Modeling:

- Exploring advanced machine learning algorithms and ensemble models can improve the accuracy and robustness of attrition predictions. Incorporating techniques such as deep learning, natural language processing (NLP), and reinforcement learning can enhance the system's predictive capabilities.
- Implementing explainable AI (XAI) techniques to provide transparent and interpretable predictions can help build trust and enable HR professionals to understand the underlying factors influencing attrition.

iii. Personalized Retention Strategies:

- Developing personalized retention strategies based on individual employee profiles and preferences can increase the effectiveness of interventions. By leveraging the insights generated by the TalentSaver system, organizations can tailor retention initiatives to address specific needs and concerns.

- Implementing feedback loops to continuously gather user input and refine the system's recommendations can ensure that retention strategies remain relevant and effective over time.

iv. Real-time Monitoring and Alerts:

- Implementing real-time monitoring and alert mechanisms can enable proactive intervention when signs of potential attrition are detected. By providing timely notifications to HR professionals and managers, the system can facilitate early engagement and mitigate the risk of turnover.
- Developing a dashboard with visualizations and key performance indicators (KPIs) can help organizations track attrition trends and measure the impact of retention efforts.

v. Cross-organizational Learning and Benchmarking:

- Collaborating with multiple organizations to create a shared knowledge base and benchmarking framework can provide valuable insights into industry-specific attrition patterns and best practices. By leveraging aggregated data from different organizations, the TalentSaver system can identify common attrition drivers and effective retention strategies.
- Establishing partnerships with academic institutions and research organizations can facilitate ongoing research and innovation in the field of employee attrition prediction and management.

8.5 Conclusion:

In conclusion, the TalentSaver project has demonstrated significant potential in addressing employee attrition through advanced machine learning techniques. By continuously evolving and addressing the identified challenges, the system can become a vital tool for organizations seeking to retain their valuable talent and foster a positive work environment.

Chapter # 9 Complete Code Of Final Year Project

9.1 Backend

9.1.1 Main source code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn import datasets
from sklearn.metrics import accuracy_score

attrdata = pd.read_csv("Table_1.csv")

attrdata.head()

attrdata.drop(0,inplace=True)
attrdata.isnull().sum()

attrdata.dropna(axis=0,inplace=True)

attrdata.shape

gender_dict = attrdata["Gender "].value_counts()
gender_dict

attrdata['Gender '].value_counts().plot(kind='bar',color=['salmon','lightblue'],title="Count of
different gender")

#Create a plot for crosstab

pd.crosstab(attrdata['Gender '],attrdata['Stay/Left']).plot(kind="bar",figsize=(10,6))
plt.title("Stay/Left vs Gender")
plt.xlabel("Stay/Left")
plt.ylabel("No of people who left based on gender")
plt.legend(["Left","Stay"])
plt.xticks(rotation=0)

promoted_dict = attrdata["Promoted/Non Promoted"].value_counts()
promoted_dict
```

```
attrdata['Promoted/Non
Promoted'].value_counts().plot(kind='bar',color=['salmon','lightblue'],title="Promoted and Non
Promoted")

#Create a plot for crosstab

pd.crosstab(attrdata['Promoted/Non
Promoted'],attrdata['Stay/Left']).plot(kind="bar",figsize=(10,6))
plt.title("Stay/Left vs Promoted/Non Promoted")
plt.xlabel("Stay/Left")
plt.ylabel("No. of people who left/stay based on promotion")
plt.legend(["Left","Stay"])
plt.xticks(rotation=0)

func_dict = attrdata["Function"].value_counts()
func_dict

attrdata['Function'].value_counts().plot(kind='bar',color=['salmon','lightblue'],title="Functions in
organization")

#Create a plot for crosstab

pd.crosstab(attrdata['Function'],attrdata['Stay/Left']).plot(kind="bar",figsize=(10,6))
plt.title("Stay/Left vs Function")
plt.xlabel("Stay/Left")
plt.ylabel("No. of people who left/stay based on function of organization")
plt.legend(["Left","Stay"])
plt.xticks(rotation=0)

Marital_dict = attrdata["Marital Status"].value_counts()
print(Marital_dict)

Emp_dict = attrdata["Emp. Group"].value_counts()
Emp_dict['other group'] = 1
print(Emp_dict)

job_dict = attrdata["Job Role Match"].value_counts()
job_dict

attrdata['Job Role Match'].value_counts().plot(kind='bar',color=['salmon','lightblue'],title="Job
Role Match")

#Create a plot for crosstab
```

```
pd.crosstab(attrdata['Job Role Match'],attrdata['Stay/Left']).plot(kind="bar",figsize=(10,6))
plt.title("Stay/Left vs Job Role Match")
plt.xlabel("Stay/Left")
plt.legend(["Left","Stay"])
plt.xticks(rotation=0)

tenure_dict = attrdata["Tenure Grp."].value_counts()
print(tenure_dict)

# Its Age vs stay/left
import seaborn as sns
sns.jointplot(x='Stay/Left',y='Age in YY.',data=attrdata)

sns.jointplot(x='Stay/Left',y='Experience (YY.MM)',data=attrdata)


sns.jointplot(x='Stay/Left',y='Experience (YY.MM)',data=attrdata)


# Let's make our correlation matrix visual

# Step 1: Select only numeric columns
numeric_attrdata = attrdata.select_dtypes(include=[np.number])

# Step 2: Calculate the correlation matrix
corr_matrix = numeric_attrdata.corr()

# Step 3: Plot the correlation matrix
fig, ax = plt.subplots(figsize=(15, 10))
ax = sns.heatmap(corr_matrix, annot=True, linewidths=0.5, fmt=".2f")
plt.show()

location_dict = attrdata["Location"].value_counts()
print(location_dict)

location_dict_new = {
    'Chennai':       7,
    'Noida':         6,
    'Bangalore':     5,
    'Hyderabad':     4,
    'Pune':          3,
    'Madurai':       2,
```

```
    'Lucknow':       1,
    'other place':   0,
}

print(location_dict_new)

def location(x):
    if str(x) in location_dict_new.keys():
        return location_dict_new[str(x)]
    else:
        return location_dict_new['other place']

data_l = attrdata["Location"].apply(location)
attrdata['New Location'] = data_l
attrdata.head()

gen = pd.get_dummies(attrdata["Function"])
gen.head()

hr = pd.get_dummies(attrdata["Hiring Source"])
hr.head()

def Mar(x):
    if str(x) in Marital_dict.keys() and Marital_dict[str(x)] > 100:
        return str(x)
    else:
        return 'other status'

data_l = attrdata["Marital Status"].apply(Mar)
attrdata['New Marital'] = data_l
attrdata.head()

Mr = pd.get_dummies(attrdata["New Marital"])
Mr.head()

def Promoted(x):
    if x == 'Promoted':
        return int(1)
    else:
        return int(0)

data_l = attrdata["Promoted/Non Promoted"].apply(Promoted)
attrdata['New Promotion'] = data_l
```

```
attrdata.head()

Emp_dict_new = {
    'B1': 4,
    'B2': 3,
    'B3': 2,
    'other group': 1,
}
def emp(x):
    if str(x) in Emp_dict_new.keys():
        return str(x)
    else:
        return 'other group'

data_l = attrdata["Emp. Group"].apply(emp)
attrdata['New EMP'] = data_l

emp = pd.get_dummies(attrdata["New EMP"])
attrdata.head()

def Job(x):
    if x == 'Yes':
        return int(1)
    else:
        return int(0)

data_l = attrdata["Job Role Match"].apply(Job)
attrdata['New Job Role Match'] = data_l
attrdata.head()

def Gen(x):
    if x in gender_dict.keys():
        return str(x)
    else:
        return 'other'

data_l = attrdata["Gender "].apply(Gen)
attrdata['New Gender'] = data_l
attrdata.head()

gend = pd.get_dummies(attrdata["New Gender"])
gend.head()
```

```
tengrp = pd.get_dummies(attrdata["Tenure Grp."])
tengrp.head()

dataset = pd.concat([attrdata, hr, Mr, emp, tengrp, gen, gend], axis = 1)
dataset.head()

dataset.columns

dataset.drop(["table id", "name", "Marital Status","Promoted/Non Promoted","Function","Emp.
Group","Job Role Match","Location"
            ,"Hiring Source","Gender ", 'Tenure', 'New Gender', 'New Marital', 'New
EMP'],axis=1,inplace=True)

dataset1 = dataset.drop(['Tenure Grp.', 'phone number'], axis = 1)
dataset1.columns

# Let's make our correlation matrix visual

# Step 1: Select only numeric columns
numeric_attrdata = attrdata.select_dtypes(include=[np.number])

# Step 2: Calculate the correlation matrix
corr_matrix = numeric_attrdata.corr()

# Step 3: Plot the correlation matrix
fig, ax = plt.subplots(figsize=(15, 10))
ax = sns.heatmap(corr_matrix, annot=True, linewidths=0.5, fmt=".2f")
plt.show()

# Target
"""
def Target(x):
    if x in "Stay":
        return False
    else:
        return True

data_l = dataset1["Stay/Left"].apply(Target)
dataset1['Stay/Left'] = data_l
"""
dataset1['Stay/Left'].head()

dataset1.to_csv("processed table.csv")
```

```
dataset = pd.read_csv("processed table.csv")
dataset = pd.DataFrame(dataset)
y = dataset["Stay/Left"]
X = dataset.drop("Stay/Left",axis=1)

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=4)
X_train.head()

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn import svm

lr=LogisticRegression(C = 0.1, random_state = 42, solver = 'liblinear')
dt=DecisionTreeClassifier()
rm=RandomForestClassifier()
gnb=GaussianNB()
knn = KNeighborsClassifier(n_neighbors=3)
svm = svm.SVC(kernel='linear')

for a,b in zip([lr,dt,knn,svm,rm,gnb],["Logistic Regression","Decision
Tree","KNN","SVM","Random Forest","Naive Bayes"]):
    a.fit(X_train,y_train)
    prediction=a.predict(X_train)
    y_pred=a.predict(X_test)
    score1=accuracy_score(y_train,prediction)
    score=accuracy_score(y_test,y_pred)
    msg1="[%s] training data accuracy is : %f" % (b,score1)
    msg2="[%s] test data accuracy is : %f" % (b,score)
    print(msg1)
    print(msg2)

model_scores={'Logistic Regression':lr.score(X_test,y_test),
            'KNN classifier':knn.score(X_test,y_test),
            'Support Vector Machine':svm.score(X_test,y_test),
            'Random forest':rm.score(X_test,y_test),
             'Decision tree':dt.score(X_test,y_test),
             'Naive Bayes':gnb.score(X_test,y_test)
            }
model_scores
```

```
from sklearn.metrics import classification_report

rm_y_preds = rm.predict(X_test)

print(classification_report(y_test,rm_y_preds))

from sklearn.metrics import classification_report

lr_y_preds = lr.predict(X_test)

print(classification_report(y_test,lr_y_preds))

model_compare=pd.DataFrame(model_scores,index=['accuracy'])
model_compare

model_compare.T.plot(kind='bar') # (T is here for transpose)

# Logistic regression
feature_dict=dict(zip(dataset.columns,list(lr.coef_[0])))
feature_dict

feature_df=pd.DataFrame(feature_dict,index=[0])
feature_df.T.plot(kind="bar",legend=False,title="Feature Importance")

import pickle

# Save the trained model as a pickle string.
saved_model = pickle.dumps(lr)

# Load the pickled model
lr_from_pickle = pickle.loads(saved_model)

# Use the loaded pickled model to make predictions
lr_from_pickle.predict(X_test)

# loading dependency
import joblib

# saving our model - model - model , filename - model_lr
joblib.dump(lr , 'model_lr')

# opening the file- model_jlib
```

```
m_jlib = joblib.load('model_lr')

# check prediction
m_jlib.predict(X_test) # similar output

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.neighbors import NearestNeighbors

# Step 1: Data Preprocessing
# Load the data
data = pd.read_csv('Table_1.csv')

# Strip any extra spaces from the column names
data.columns = data.columns.str.strip()

# Handle missing values (if any)
data.fillna(method='ffill', inplace=True)

# Encode categorical variables
label_encoders = {}
for column in ['Location', 'Emp. Group', 'Function', 'Gender', 'Tenure Grp.', 'Marital Status',
'Hiring Source', 'Promoted/Non Promoted', 'Job Role Match', 'Stay/Left']:
    le = LabelEncoder()
    data[column] = le.fit_transform(data[column].astype(str))
    label_encoders[column] = le

# Separate features and target variable
X = data.drop(columns=['Stay/Left', 'name', 'phone number', 'table id'])
y = data['Stay/Left']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 2: Model Training
```

```
# Train a RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate the model
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# Step 3: Replacement Selection
def find_best_replacement(leaving_employee_id, data, model, scaler):


  # Get the details of the employee who is leaving
    leaving_employee = data.loc[data['table id'] ==
leaving_employee_id].drop(columns=['Stay/Left', 'name', 'phone number', 'table id'])

    # Scale the leaving employee's data
    leaving_employee_scaled = scaler.transform(leaving_employee)

    # Use NearestNeighbors to find the most similar employees
    knn = NearestNeighbors(n_neighbors=5)
    knn.fit(X)
    distances, indices = knn.kneighbors(leaving_employee_scaled)

    # Get the top candidates
    top_candidates = data.iloc[indices[0]]
    top_candidates = top_candidates[top_candidates['Stay/Left'] ==
label_encoders['Stay/Left'].transform(['Stay'])[0]]

    return top_candidates

# Example: Find replacement for employee with table id 1
leaving_employee_id = 1
best_replacements = find_best_replacement(leaving_employee_id, data, model, scaler)
print("Best Replacements for Employee ID 1:\n", best_replacements)
```

9.1.2 App.py

```
from flask import Flask, request, render_template, jsonify, redirect, url_for, flash, session

from flask_sqlalchemy import SQLAlchemy
```

```
from werkzeug.security import generate_password_hash, check_password_hash
import os
from dotenv import load_dotenv
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import NearestNeighbors
from functools import wraps

# Load environment variables from .env file
load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY')
app.config['UPLOAD_FOLDER'] = 'uploads'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

# User model
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)

# Ensure the upload folder exists
```

```
if not os.path.exists(app.config['UPLOAD_FOLDER']):
    os.makedirs(app.config['UPLOAD_FOLDER'])

# Function to load and process the CSV file
def load_and_process_csv(filepath):
    data = pd.read_csv(filepath)
    data.columns = data.columns.str.strip()
    data.fillna(method='ffill', inplace=True)

    label_encoders = {}
    for column in ['Location', 'Emp. Group', 'Function', 'Gender', 'Tenure Grp.', 'Marital Status',
'Hiring Source', 'Promoted/Non Promoted', 'Job Role Match', 'Stay/Left']:
        le = LabelEncoder()
        data[column] = le.fit_transform(data[column].astype(str))
        label_encoders[column] = le

    X = data.drop(columns=['Stay/Left', 'name', 'phone number', 'table id'])
    y = data['Stay/Left']

    scaler = StandardScaler()
    X = scaler.fit_transform(X)

    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X, y)

    return data, X, y, scaler, model, label_encoders

# Login required decorator
def login_required(f):
```

```
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            flash('Please log in to access this page.', 'warning')
            return redirect(url_for('login'))
        return f(*args, **kwargs)
    return decorated_function

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        username = request.form.get('username')
        email = request.form.get('email')
        password = request.form.get('password')
        hashed_password = generate_password_hash(password, method='pbkdf2:sha256')

        new_user = User(username=username, email=email, password=hashed_password)
        db.session.add(new_user)
        db.session.commit()
```

```
        flash('Signup successful!', 'success')
        return redirect(url_for('login'))

    return render_template('signup.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')

        user = User.query.filter_by(email=email).first()
        if user and check_password_hash(user.password, password):
            session['user_id'] = user.id
            session['username'] = user.username
            flash('Login successful!', 'success')
            return redirect(url_for('home'))
        else:
            flash('Login failed. Check your email and password.', 'danger')

    return render_template('login.html')

@app.route('/logout')
@login_required
def logout():
    session.clear()
    flash('You have been logged out.', 'success')
    return redirect(url_for('login'))
```

```
@app.route('/results')
@login_required
def results():
    return render_template('results.html')

@app.route('/add_employee')
@login_required
def add_employee():
    return render_template('add_employee.html')

@app.route('/upload_csv', methods=['POST'])
@login_required
def upload_csv():
    if 'csvFile' not in request.files:
        return jsonify({'error': 'No file part'}), 400

    file = request.files['csvFile']
    if file.filename == '':
        return jsonify({'error': 'No selected file'}), 400

    if file and file.filename.endswith('.csv'):
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
        file.save(filepath)

        # Load and process the new CSV file
        global data, X, y, scaler, model, label_encoders
        data, X, y, scaler, model, label_encoders = load_and_process_csv(filepath)
```

```
        return jsonify({'filepath': filepath}), 200

    return jsonify({'error': 'Invalid file type'}), 400

@app.route('/find_replacement', methods=['POST'])
@login_required
def find_replacement():
    employee_id = int(request.form['employee_id'])
    leaving_employee = data.loc[data['table id'] == employee_id].drop(columns=['Stay/Left',
'name', 'phone number', 'table id'])

    # Scale the leaving employee's data
    leaving_employee_scaled = scaler.transform(leaving_employee)

    # Use NearestNeighbors to find the most similar employees
    knn = NearestNeighbors(n_neighbors=5)
    knn.fit(X)
    distances, indices = knn.kneighbors(leaving_employee_scaled)

    # Get the top candidates
    top_candidates = data.iloc[indices[0]]
    top_candidates = top_candidates[top_candidates['Stay/Left'] ==
label_encoders['Stay/Left'].transform(['Stay'])[0]]

    # Select necessary columns for output
    output_columns = ['table id', 'name', 'Age in YY.', 'Location', 'Emp. Group', 'Function',
'Gender', 'Tenure', 'Experience (YY.MM)', 'Marital Status', 'Hiring Source', 'Promoted/Non
Promoted', 'Job Role Match']
    top_candidates = top_candidates[output_columns]
```

```
    return jsonify(top_candidates.to_dict(orient='records'))

if __name__ == '__main__':
    app.run(debug=True)
```

9.1.3 Requirementst.txt

```
pandas
numpy
matplotlib
scikit-learn
seaborn
flask
flask_sqlalchemy
werkzeug
python-dotenv
```

9.2 Frontend

9.2.1 Index

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>Home</title>

  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"
integrity="sha384-
Vkoo8x4CGsO3+Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh"
crossorigin="anonymous">

  <style>

    body {

  background: linear-gradient(45deg, #3c6478, #3c6478, #3c6478);



      color: white;

    }





    /* Gradient animation */

    @keyframes gradientAnimation {

      0% { background-position: 0% 50%; }

      50% { background-position: 100% 50%; }

      100% { background-position: 0% 50%; }

    }

    </style>
```

```
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">TalentSaver</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link" href="/">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/about">About</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/results">Check Replacements</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/login">Login</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/logout">logout</a>
          </li>
        </ul>
      </div>
```

```
    </div>
  </nav>
  <div class="container mt-5">
    <div align="center">
    <img src="\static\2ND.png" height="250" width="250">
    </div>
    <h1>Welcome to TalentSaver</h1>
    <div class="row">
    <div class="col">
      <p>This Flask web application is designed to manage user authentication and facilitate employee data processing through machine learning techniques. It features user registration and login functionalities secured by hashed passwords. Once logged in, users can upload employee data in CSV format, which is processed to predict employee retention using a RandomForestClassifier model. The application also supports finding suitable replacement candidates for employees who leave the organization by leveraging the NearestNeighbors algorithm. The code ensures data integrity by using SQLAlchemy for database interactions, with a user model defined for managing user credentials and sessions. The application includes several routes to render different HTML templates for user interactions, upload files, and display results.</p>
    </div>


    <div class="col">
      <img src="\static\1.png" height="300" width="600" >


    </div>
  </div>
  </div>
</body>
</html>
```

9.2.2 Login

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>login</title>

  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"
integrity="sha384-
Vkoo8x4CGsO3+Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh"
crossorigin="anonymous">

<style>

  body {

  background: linear-gradient(45deg, #3c6478, #3c6478, #3c6478);


  color: white;

  }



  /* Gradient animation */

  @keyframes gradientAnimation {

    0% { background-position: 0% 50%; }

    50% { background-position: 100% 50%; }

    100% { background-position: 0% 50%; }

  }

  p{

    font-size:18px;

  }

  </style>

</head>
```

```
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
<div class="container-fluid">
  <a class="navbar-brand" href="#">TalentSaver</a>
  <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav">
      <li class="nav-item">
        <a class="nav-link" href="/">Home</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="/about">About</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="/results">Check Replacements</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="/login">Login</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="/logout">logout</a>
      </li>
    </ul>
  </div>
</div>
```

```
</nav>
<div class="container">
  <div class="row justify-content-center">
      <div class="col-md-6">
          <h2 class="text-center">Login</h2>
          <form method="post" action="/login">
              <div class="form-group">
                  <label for="email">Email address</label>
                  <input type="email" class="form-control" id="email" name="email" required>
              </div>
              <div class="form-group">
                  <label for="password">Password</label>
                  <input type="password" class="form-control" id="password" name="password"
required>
              </div>
              <button type="submit" class="btn btn-primary btn-block">Login</button>
          </form>
      </div>
  </div>
</div><br><br>
<center>
<p>don't have any account? <a href="/signup">signup here</a></p>
</center>
</body>
</html>
```

9.2.3 Signup

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>signup</title>

  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"
integrity="sha384-
Vkoo8x4CGsO3+Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh"
crossorigin="anonymous">

  <style>

    body {

  background: linear-gradient(45deg, #3c6478, #3c6478, #3c6478);

  color: white;




    }


    /* Gradient animation */

    @keyframes gradientAnimation {

      0% { background-position: 0% 50%; }

      50% { background-position: 100% 50%; }

      100% { background-position: 0% 50%; }

    }

    p{

      font-size:18px;

    }

    </style>

</head>
```

```
  <body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">TalentSaver</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle
navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link" href="/">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/about">About</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/results">Check Replacements</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/login">Login</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/logout">logout</a>
          </li>
        </ul>
      </div>
```

```
    </div>
  </nav>
  <div class="container">
    <div class="container">
      <div class="row justify-content-center">
          <div class="col-md-6">
              <h2 class="text-center">Signup</h2>
              <form method="post" action="/signup">
                  <div class="form-group">
                      <label for="username">Username</label>
                      <input type="text" class="form-control" id="username" name="username"
required>
                  </div>
                  <div class="form-group">
                      <label for="email">Email address</label>
                      <input type="email" class="form-control" id="email" name="email" required>
                  </div>
                  <div class="form-group">
                      <label for="password">Password</label>
                      <input type="password" class="form-control" id="password" name="password"
required>
                  </div>
                  <button type="submit" class="btn btn-primary btn-block">Signup</button>
              </form>
          </div>
      </div>
  </div><br><br>
<center>
  <p>already have an account? <a href="/login">login here</a></p>
```

```
</center>
</body>
</html>
```

9.2.4 Users

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Users</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"
integrity="sha384-
Vkoo8x4CGsO3+Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh"
crossorigin="anonymous">
  <style>
    body {
  background: linear-gradient(45deg, #3c6478, #3c6478, #3c6478);


      color: white;
    }


    /* Gradient animation */
    @keyframes gradientAnimation {
      0% { background-position: 0% 50%; }
      50% { background-position: 100% 50%; }
      100% { background-position: 0% 50%; }
    }
```

```
    </style>
</head>


  <body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">TalentSaver</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle
navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link" href="/">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/about">About</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/results">Check Replacements</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/login">Login</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/logout">logout</a>
          </li>
```

```
        </ul>
      </div>
    </div>
  </nav>

  <div class="container mt-5">
    <h1>Welcome to TalentSaver</h1>
    <p>This is the users page.</p>
  </div>
</body>
</html>
```

9.2.5 About

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"
integrity="sha384-
Vkoo8x4CGsO3+Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh"
crossorigin="anonymous">
<style>
 b{
  font-size:20px;
 }
```

```
/* Gradient background */
body {
  background: linear-gradient(45deg, #3c6478, #3c6478, #3c6478);
  color: white;
}

/* Gradient animation */
@keyframes gradientAnimation {
  0% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
  100% { background-position: 0% 50%; }
}
</style>
</head>
<body>

  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">TalentSaver</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link" href="/">Home</a>
          </li>
```

```
        <li class="nav-item">
          <a class="nav-link" href="/about">About</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="/results">Check Replacements</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="/login">Login</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="/logout">logout</a>
        </li>
      </ul>
    </div>
  </div>
</nav>

<div class="container mt-5">

  <div class="row">
    <img src="\static\2.jpg" width="1120" height="330" >
  </div>
  <h1>Welcome to TalentSaver</h1>
  <br>

    <div class="row">
      <p>This Flask web application is a comprehensive system designed for user authentication and advanced employee data processing using machine learning techniques. It encompasses several functionalities that work together to provide a seamless experience for managing user
```

registrations, logins, and employee data analysis. Below, we'll delve into the key components and features of this application.

User Authentication

The application begins with essential user authentication features. It allows users to sign up, log in, and log out, with all credentials securely managed using hashed passwords. The `User` model is defined using SQLAlchemy, which includes fields for `username`, `email`, and `password`. When a user registers, their password is hashed using the `werkzeug.security.generate_password_hash` function for secure storage. The application uses session management to keep track of logged-in users and implements a `login_required` decorator to protect routes that should only be accessible to authenticated users.

Environment Configuration and Initialization

Environment variables are loaded using the `dotenv` package, which ensures sensitive information such as the application's secret key is securely managed. The application is configured to use a SQLite database, and it specifies an upload folder for handling CSV files.

 Upload and Process CSV Files

A core functionality of the application is the ability to upload employee data via CSV files. The `upload_csv` route handles the file upload, ensuring the file is valid and saving it to the designated folder. Once a file is uploaded, it is processed by the `load_and_process_csv` function. This function reads the CSV file into a pandas DataFrame, cleans and preprocesses the data by stripping column names, filling missing values, and encoding categorical variables using `LabelEncoder`. It then separates the features from the target variable, scales the feature set using `StandardScaler`, and trains a `RandomForestClassifier` model to predict whether an employee will stay or leave the company.

Employee Data Analysis and Prediction

The application is designed to analyze employee data and provide actionable insights. By leveraging the trained RandomForest model, the application can make predictions about employee retention. Users can query the model to find out which employees are likely to stay or leave based on their characteristics and historical data.

 Finding Replacement Candidates

A unique feature of this application is its ability to find suitable replacement candidates for employees who leave the organization. When an employee decides to leave, users can use the `find_replacement` route to identify potential replacements. This is accomplished using the `NearestNeighbors` algorithm, which finds the most similar employees to the departing one. The application scales the departing employee's data, searches for similar profiles among current employees, and filters the results to include only those likely to stay, as indicated by the prediction model. The output is a list of top candidates with relevant details such as their ID, name, age, location, employee group, function, gender, tenure, experience, marital status, hiring source, promotion status, and job role match.

 User Interface and Interaction

The user interface is built using HTML templates rendered by Flask. The application includes routes for the home page, about page, signup page, login page, results page, and a page for adding employee details. Each route renders the appropriate template and handles the necessary logic for user interaction. For instance, the signup and login routes handle form submissions, validate input, and provide feedback to the user through flash messages.

 Session Management and Security

Security is a critical aspect of this application. Sessions are used to maintain user state, ensuring that users remain logged in across different pages. The `login_required` decorator is applied to routes that should only be accessible to authenticated users, redirecting unauthenticated users to the login page with a warning message. This ensures that sensitive operations, such as uploading CSV files and finding replacement candidates, are protected.

 Scalability and Maintenance

The application is designed with scalability in mind. The modular structure of the code, use of environment variables, and SQLAlchemy for database interactions ensure that the application can be easily maintained and scaled. The data processing functions are designed to handle various employee datasets, making it adaptable to different organizational needs.

Conclusion

In summary, this Flask web application provides a robust platform for managing user authentication and employee data analysis. By integrating machine learning models, it offers predictive insights into employee retention and helps identify suitable replacement candidates. The secure, user-friendly interface and scalable design make it a valuable tool for organizations looking to optimize their workforce management processes.</p>

```
    </div>
  </div>
</body>
</html>
```

9.2.6 Add_Employee

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Add HR</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"
integrity="sha384-
Vkoo8x4CGsO3+Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh"
crossorigin="anonymous"></head>
  <style>
    body {
      background-color: #3c6478;
      color: white;
    }
  </style>
<body><nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">TalentSaver</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle
navigation">
        <span class="navbar-toggler-icon"></span>
```

```
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link" href="/">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/about">About</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/results">Check Replacements</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/login">Login</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/logout">logout</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>

  <div class="container mt-5">
    <h1>Welcome to TalentSaver</h1>
    <p>Add New HR</p>
  </div>
</body>
```

```
</html>
```

9.2.7 Navbar

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bootstrap Navbar</title>
  <link href="https://stackpath.bootstrapcdn.com/bootstrap/5.1.3/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">Navbar</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNavDropdown" aria-controls="navbarNavDropdown" aria-expanded="false"
aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNavDropdown">
      <ul class="navbar-nav">
        <li class="nav-item">
          <a class="nav-link active" aria-current="page" href="#">Home</a>
        </li>
        <li class="nav-item">
```

```
          <a class="nav-link" href="#">Features</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Pricing</a>
        </li>
        <li class="nav-item dropdown">
          <a class="nav-link dropdown-toggle" href="#" id="navbarDropdownMenuLink"
role="button" data-bs-toggle="dropdown" aria-expanded="false">
            Dropdown
          </a>
          <ul class="dropdown-menu" aria-labelledby="navbarDropdownMenuLink">
            <li><a class="dropdown-item" href="#">Action</a></li>
            <li><a class="dropdown-item" href="#">Another action</a></li>
            <li><a class="dropdown-item" href="#">Something else here</a></li>
          </ul>
        </li>
      </ul>
    </div>
  </div>
</nav>

<script
src="https://stackpath.bootstrapcdn.com/bootstrap/5.1.3/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

9.2.8 Results

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Employee Replacement Finder</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">

    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"
integrity="sha384-
Vkoo8x4CGsO3+Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh"
crossorigin="anonymous">

    <style>

        body {

  background: linear-gradient(45deg, #3c6478, #3c6478, #3c6478);


          color: white;

        }


    .btn.btn-outline-secondary {

    background-color: black;

    color: white;

    border: 1px solid black;


        /* Gradient animation */

        @keyframes gradientAnimation {

          0% { background-position: 0% 50%; }

          50% { background-position: 100% 50%; }

          100% { background-position: 0% 50%; }
```

```
        }
        </style>
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
        <div class="container-fluid">
          <a class="navbar-brand" href="#">TalentSaver</a>
          <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle
navigation">
            <span class="navbar-toggler-icon"></span>
          </button>
          <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav">
                <li class="nav-item">
                  <a class="nav-link" href="/">Home</a>
                </li>
                <li class="nav-item">
                  <a class="nav-link" href="/about">About</a>
                </li>
                <li class="nav-item">
                  <a class="nav-link" href="/results">Check Replacements</a>
                </li>
                <li class="nav-item">
                  <a class="nav-link" href="/login">Login</a>
                </li>
                <li class="nav-item">
                  <a class="nav-link" href="/logout">logout</a>
                </li>
```

```
            </ul>
          </div>
        </div>
      </nav>


    <div class="container mt-4 pt-4">
        <h1>Find Best Replacement for Leaving Employee</h1>
        <div class="input-group mb-3">
            <form id="upload-form" method="POST" enctype="multipart/form-data" class="w-
100">
                <div class="input-group">
                    <div class="custom-file">
                        <input type="file" class="custom-file-input" id="csvFile" name="csvFile"
accept=".csv" required>
                        <label class="custom-file-label" for="csvFile">Choose file</label>
                    </div>
                    <div class="input-group-append">
                        <button class="btn btn-outline-secondary" type="submit">Upload</button>
                    </div>
                </div>
            </form>
        </div>
        <div id="file-path" class="mt-2"></div>
        <br><br>
        <form action="/find_replacement" method="post" id="replacementForm">
            <div class="form-group">
                <label for="employee_id">Enter Employee ID:</label>
                <input type="number" class="form-control col-sm-3" id="employee_id"
name="employee_id" required>
```

```
        </div>
        <button type="submit" class="btn btn-dark">Find Replacement</button>
    </form>
    <br><br>
    <div id="result"></div>
</div>

<script>
    document.getElementById('upload-form').onsubmit = async function(event) {
        event.preventDefault();
        let formData = new FormData(this);
        let response = await fetch('/upload_csv', {
            method: 'POST',
            body: formData
        });
        let result = await response.json();
        let filePathDiv = document.getElementById('file-path');
        if (result.filepath) {
            filePathDiv.innerHTML = `<p>File uploaded successfully! File path:
${result.filepath}</p>`;
        } else {
            filePathDiv.innerHTML = `<p>Error uploading file.</p>`;
        }
    };

    document.getElementById('replacementForm').onsubmit = async function(event) {
        event.preventDefault();
        let formData = new FormData(this);
        let response = await fetch('/find_replacement', {
```

```
            method: 'POST',
            body: formData
        });
        let result = await response.json();
        let resultDiv = document.getElementById('result');

        if (result.length > 0) {
            let table = '<h2>Best Replacements:</h2><br><table class="table"><thead><tr>';
            let keys = Object.keys(result[0]);
            for (let key of keys) {
                table += '<th>' + key + '</th>';
            }
            table += '</tr></thead><tbody>';
            for (let row of result) {
                table += '<tr>';
                for (let key of keys) {
                    table += '<td>' + row[key] + '</td>';
                }
                table += '</tr>';
            }
            table += '</tbody></table>';
            resultDiv.innerHTML = table;
        } else {
            resultDiv.innerHTML = '<h2>No suitable replacements found.</h2>';
        }
    };
  </script>
</body></html>
```

References

- https://www.akkio.com/post/end-to-end-machine-learning-workflow
- https://www.clearpeaks.com/predicting-employee-attrition-with-machine-learning-using-knime/
- https://www.hindawi.com/journals/cin/2022/7728668/#references
- https://www.analyticsvidhya.com/blog/2021/11/employee-attrition-prediction-a-comprehensive-guide/
- https://www.researchgate.net/publication/332528099_Early_Prediction_of_Employee Attrition using Data Mining Techniques
- https://www.warse.org/IJATCSE/static/pdf/file/ijatcse91952020.pdf
- https://www.mdpi.com/2076-3417/12/13/6424
- https://www.globalscientificjournal.com/researchpaper/Predicting_Employee_attrition_using_Decision_tree_algorithm.pdf

SUPERVISOR SIGNATURE